

GAVS: Graph-Aware Scheduling for Leakage-Resilient VOLE-in-the-Head Signatures

Anonymous Author(s)

Anonymous Institution
anonymous@example.org

Abstract. VOLE-in-the-Head (VOLEitH) signatures such as SDitH v2.0 obtain compact post-quantum transcripts by folding many algebraic checks into structured proof objects: regular syndrome decoding over $\mathbb{F}_2$, tensor-product witness compression, Batch Line Commitment (BLC) with Gray-code traversal, Mux-tree polynomial evaluation, and seed-expanded pseudorandom material. These structures, however, also create a physical-security challenge: the signer repeatedly manipulates algebraically related witness coordinates, VOLE tags, accumulator states, and selector-dependent field elements, which a soft analytical side-channel adversary can aggregate through the corresponding factor graph. This paper proposes **GAVS**, a graph-aware scheduling framework for leakage-resilient VOLEitH signing hardware, developed through a case study on SDitH v2.0. **GAVS** treats the attacker’s leakage factor graph as a first-class hardware-design input. It introduces three scheduling primitives—a randomized hypercube-automorphic Gray traversal for BLC, a two-dimensional VOLE-tag scheduler, and a Tensor-Mux scheduler for compressed witness evaluation—together with a schedule-seed-independent canonicalization buffer based on public-index Beneš routing. The verifier-visible SDitH v2.0 transcript is preserved bit-for-bit. We establish transcript equivalence, constant-work execution, and a three-layer composability stack with masking (Probe-Isolating Non-Interference, glitch- and transition-extended robust probing, and physical share-domain segregation) through a seven-theorem formal package with complete proofs. The label-classification advantage of any side-channel classifier is bounded by an M -class Top-1 inequality, complemented by a Shannon-entropy bound obtained via the Audenaert–Fannes continuity inequality and a min-entropy bound matched to the operational side-channel security metric. We present an FPGA/ASIC-oriented hardware architecture and a seven-experiment evaluation protocol aligned with ISO/IEC 17825, comparing **GAVS** against unprotected, masked, and generically shuffled baselines.

Keywords: Post-quantum signatures · VOLE-in-the-Head · SDitH · Side-channel countermeasures · Soft analytical side-channel attacks · Hardware scheduling.

1 Introduction

The NIST additional-signature process is fielding VOLE-in-the-Head (VOLEitH) candidates such as SDitH v2.0 and FAEST as a counterpart to the recently stan-

standardized lattice and hash-based signatures of FIPS 204 and FIPS 205 [38,40,39,41]; both VOLEitH candidates target hardware deployment and must resist soft analytical side-channel attacks (SASCA) [51,52]. SDitH v2.0 is a code-based Fiat-Shamir signature scheme that combines regular syndrome decoding over $\mathbb{F}_2$, tensor-product witness compression, Batch Line Commitment (BLC) with Gray-code traversal, Mux-tree polynomial evaluation, and AES/Rijndael-based seed expansion [49,17]. The algorithmic structures that make SDitH v2.0 compact also make its hardware signer a rich attack surface: the signer repeatedly manipulates algebraically related witness coordinates, VOLE tags, accumulator states, and selector-dependent field elements across a highly structured computation graph.

This paper investigates whether the execution *schedule* of a VOLEitH signer can be treated as a first-class security object. Existing implementation countermeasures for post-quantum hardware focus primarily on reducing local leakage by masking, randomizing operation order by generic shuffling, or inserting microarchitectural noise [53,36]. These techniques are necessary but not sufficient for VOLEitH-style signatures. A SASCA adversary need not recover every intermediate value independently: noisy local leakage is combined with the algebraic constraints of the algorithm through a factor graph, and recent exact-inference variants based on tractable probabilistic circuits further sharpen this threat [52]. Reported attacks on MPCitH and VOLEitH signers, including masked FAEST [30,21], confirm that the redundancy that improves proof efficiency can become a powerful source of leakage aggregation, and analogous results on HQC [23] and on shuffled NTT implementations of CRYSTALS-Kyber [25] confirm that a uniformly random shuffle can be inverted from side-channel observations.

We propose **GAVS**, a graph-aware scheduler for leakage-resilient VOLEitH signing hardware. **GAVS** does not change the SDitH v2.0 protocol, public transcript, verification algorithm, or cryptographic assumption. It changes how the signer internally orders, places, and canonicalizes computations. The signer is modeled simultaneously as a dependency DAG over hardware operations and as a leakage factor graph over witness bits, tensor coordinates, VOLE tags, BLC accumulators, mask shares, and extension-field elements. A schedule maps each operation to a time slot, hardware lane, memory bank, share domain, and canonical output index. The security goal is not only to hide absolute time. **GAVS** aims to reduce short-range repeated exposure of the same secret-dependent variables, to increase the entropy of the hidden mapping between leakage windows and factor nodes, and to preserve compatibility with masked arithmetic and constant-time execution.

The design principle is that the attacker’s inference graph should be compiled before the hardware schedule is fixed. For BLC, we introduce a randomized Gray traversal based on automorphisms of the hypercube. This preserves the one-bit-transition property that enables shift-and-XOR hardware, while randomizing the association between physical time and Gray-code dimensions. For VOLE-tag generation, we introduce a two-dimensional graph-aware scheduler that separates

row-wise and column-wise exposures of the same witness-dependent variables across time, lanes, and memory banks. For tensor-product witness compression and Mux-tree polynomial evaluation, we introduce a Tensor-Mux schedule that prevents coordinates derived from the same witness block from appearing in easily alignable neighboring cycles. A canonicalization buffer, implemented as a Beneš network driven by *public canonical indices only*, restores the standard SDitH transcript order before hashing and output. The external address bus of the canonicalizer is independent of the schedule seed by construction; the control-wire layer is bounded empirically by canonicalizer-aligned experiments in Section 8.

This paper makes the following contributions.

1. **A leakage factor-graph model for SDitH v2.0 hardware.** We derive a hardware-oriented factor graph for the SDitH v2.0 signing path, covering seed expansion, tensor witness generation, BLC accumulation, VOLE-tag generation, Mux-tree evaluation, and hash/commitment serialization. The model identifies where proof-system redundancy becomes repeated physical leakage.
2. **Graph-aware scheduling as a countermeasure paradigm.** We formulate scheduling as a joint security-and-resource optimization problem. Unlike generic shuffling, GAVS uses factor-neighborhood information to reduce redundant leakage aggregation and to increase the uncertainty of the attacker’s leakage-to-factor labeling.
3. **Three protocol-preserving randomized schedulers for VOLEitH.** A randomized hypercube-automorphic Gray traversal for BLC, a two-dimensional VOLE-tag scheduler, and a Tensor-Mux scheduler for compressed witnesses. These primitives are compatible with constant-work execution and preserve the canonical SDitH v2.0 transcript.
4. **A schedule-seed-independent canonicalization buffer.** We replace the abstract permutation network of earlier informal descriptions with an explicit Beneš construction. The external address bus is independent of the schedule seed by construction; the control-wire layer is bounded empirically by canonicalizer-focused experiments in Section 8.
5. **A seven-theorem formal package with complete proofs.** The package consists of transcript-equivalence theorems for BLC, Tensor-Mux, and VOLE-Tag with a stack-wide consolidation; a three-layer masking-composability theorem stack covering Probe-Isolating Non-Interference (PINI) preservation, robust-probing implementability, and physical share-domain segregation; an address-bus / control-wire reduction theorem for the canonicalizer; and an M -class label-entropy bound proved through the Audenaert–Fannes continuity inequality together with a Le Cam Top-1 argument, complemented by a min-entropy corollary aligned with the operational side-channel security metric.
6. **An FPGA/ASIC-ready hardware architecture and evaluation protocol.** A modular accelerator with schedule PRNG, factor-graph metadata ROM, Beneš canonicalizer, banked scratch memory, and masking-aware placement; together with a seven-experiment side-channel evaluation protocol cov-

ering classical leakage tests, profiling attacks, schedule-label recovery, BP-SASCA, exact-inference subgraph analysis, and ablations against masking-only and generic-shuffling baselines.

For VOLEitH signatures, the schedule is part of the physical attack surface. GAVS treats the attacker’s inference graph as a design input and turns protocol-preserving scheduling into a measurable security layer for post-quantum signature hardware. The case study targets SDitH v2.0 (NIST PQC additional-signature Round 2); because the framework is parameterized by public factor-graph metadata extracted from the specification, it adapts mechanically to any subsequent revision of SDitH.

Organization. Section 2 reviews SDitH v2.0 and the VOLEitH signing path. Section 3 introduces the leakage factor graph and threat model. Section 4 describes the baseline attack pipeline against an unprotected SDitH v2.0 signer. Section 5 presents the GAVS scheduling algorithms. Section 6 develops the formal security analysis with complete proofs. Section 7 describes the hardware architecture, including the Beneš canonicalizer. Section 8 reports the evaluation protocol and predicted outcomes. Section 9 concludes with limitations and future work.

2 Background on SDitH v2.0 and VOLEitH

This section fixes the notation used throughout. We focus on the signing path, since GAVS changes only the internal execution order of the signer and leaves the public key, signature format, verification algorithm, and security assumption of SDitH v2.0 unchanged.

2.1 Regular Syndrome Decoding

SDitH is a code-based Fiat–Shamir signature scheme built on a zero-knowledge proof of knowledge of a syndrome-decoding witness [49,19,43]. Let $H \in \mathbb{F}_2^{(n-k_{\text{RSD}}) \times n}$ and $y \in \mathbb{F}_2^{n-k_{\text{RSD}}}$ be the public parity-check matrix and syndrome. The secret witness is $x \in \mathbb{F}_2^n$ satisfying $Hx = y$. In the regular setting used by SDitH v2.0, x decomposes into w blocks of equal length $m = n/w$,

$$x = e_0 \parallel e_1 \parallel \cdots \parallel e_{w-1}, \quad e_i \in \mathbb{F}_2^m, \quad \text{wt}(e_i) = 1.$$

Each block is a unit vector; this gives the proof system additional algebraic structure and gives the hardware signer a natural per-block organization. The hardness of recovering x reduces to standard regular syndrome decoding [6]. In hardware, the matrix H is too large to store and is expanded on demand from a public seed; the secret key seed expands into the regular witness by sampling one nonzero position per block [17].

2.2 Tensor-Product Witness Compression and Mux-Tree Evaluation

Assuming $m = \prod_{j=1}^d \mu_j$, each unit vector e_i admits the Kronecker representation

$$e_i = u_{i,1} \otimes u_{i,2} \otimes \cdots \otimes u_{i,d}, \quad u_{i,j} \in \mathbb{F}_2^{\mu_j}.$$

For each small unit vector $u_{i,j}$, we adopt the convention that the leading coordinate is not stored and is reconstructed by parity:

$$u_{i,j,0} = 1 + \sum_{\ell=1}^{\mu_j-1} u_{i,j,\ell} \pmod{2}. \quad (1)$$

The compressed witness for one block uses $\sum_{j=1}^d (\mu_j - 1)$ bits rather than m bits. Equation (1) is an encoding convention for honest one-hot witnesses; it does not by itself enforce one-hotness when $\mu_j > 2$. Soundness of the proof system relies on the unit-vector constraints in the SDitH polynomial interactive oracle proof, and the Mux formula below assumes a valid one-hot selector. Because the same coordinate of $u_{i,j}$ influences several downstream polynomial evaluations, the execution schedule becomes physically relevant: aligned leakage from multiple Mux-tree nodes that share a selector bit can be aggregated by SASCA.

The Mux-tree primitive evaluates

$$\text{Mux}_\mu(b; z_0, \dots, z_{\mu-1}) = z_0 + \sum_{\ell=1}^{\mu-1} b_\ell (z_\ell + z_0) \quad (2)$$

over $\mathbb{F}_{2^\lambda}$, where $b = (b_1, \dots, b_{\mu-1}) \in \mathbb{F}_2^{\mu-1}$ is the compressed selector drawn from Eq. (1). Under the leading-coordinate-omitted convention, an all-zero selector reconstructs $u_{i,j,0} = 1$ and selects z_0 ; a selector with $b_{\ell^*} = 1$ for some $\ell^* \geq 1$ reconstructs $u_{i,j,0} = 0$ and selects z_{ℓ^*} . For tensor-product witnesses, these multiplexers are composed across the d tensor dimensions, yielding a tree of bit-gated XOR networks over wide field elements.

2.3 VOLE-in-the-Head and the Signing Path

A vector oblivious linear evaluation (VOLE) correlation over $\mathbb{F}_{2^\lambda}$ consists of vectors $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathbb{F}_{2^\lambda}^L$ and a scalar $\Delta \in \mathbb{F}_{2^\lambda}$ satisfying $\mathbf{w} = \Delta \mathbf{u} + \mathbf{v}$. In VOLEitH, the prover simulates this correlation in its head and proves consistency of the resulting algebraic transcript [49,3,27]. The transition from MPCitH to VOLEitH reduces transcript size by increasing algebraic structure; the same structure later appears as a factor graph for side-channel inference.

The SDitH v2.0 signing path comprises six hardware-relevant macro-stages: (1) AES/Rijndael-based seed expansion; (2) regular witness expansion into tensor coordinates; (3) commitment and VOLE generation, including batched all-but-one vector commitments and BLC; (4) polynomial checks encoding the syndrome equation, witness regularity, and unit-vector constraints; (5) Fiat–Shamir

challenge derivation and response opening; and (6) canonical serialization of the signature. GAVS operates strictly inside macro-stages (2)–(4): it reorders independent operations and changes lane/memory-bank assignments, then restores canonical order before stages (5) and (6).

2.4 Batch Line Commitment and Gray-Code Traversal

BLC compresses many random field elements into a small number of accumulated values that the verifier checks later [49,17]. Let $N = 2^\kappa$ be the size of an evaluation domain and let $r_0, \dots, r_{N-1} \in \mathbb{F}_{2^\lambda}$ be pseudorandom field elements generated from public seeds, each r_u associated with a canonical domain point $u \in \{0, 1\}^\kappa$ as a public function of u . The BLC accumulators are

$$r_{\text{acc}} = \sum_{u \in \{0,1\}^\kappa} r_u, \quad r_{\text{base}} = \sum_{u \in \{0,1\}^\kappa} u \cdot r_u, \quad (3)$$

where $u \cdot r_u$ denotes coordinate-wise gating in $\mathbb{F}_{2^\lambda}^\kappa$: each of the κ bits of u selects whether the corresponding coordinate of r_u contributes to r_{base} . SDitH v2.0 uses Gray-code traversal $g_i = i \oplus (i \gg 1)$ so that consecutive domain points differ in exactly one coordinate, reducing the base accumulation to a select-and-XOR update [49]. A fixed Gray-code order, however, gives the implementation a deterministic temporal pattern: a particular coordinate transition is associated with a predictable cycle interval. Section 5.3 preserves the one-bit-transition advantage while randomizing the mapping between domain coordinates, physical time, and hardware lanes.

3 Leakage Factor Graph of VOLEitH Hardware

This section introduces the leakage model used by GAVS. The goal is to build an implementation-oriented model that connects three objects: the algebraic constraints of VOLEitH, the hardware operations that implement them, and the physical leakage samples observed by an attacker.

3.1 Two Coupled Graphs

Let $\mathcal{O} = \{o_1, \dots, o_m\}$ be the set of hardware operations executed during one SDitH v2.0 signing operation. The legal execution orders are captured by a dependency DAG $\mathcal{D}_{\text{hw}} = (\mathcal{O}, \mathcal{E}_{\text{dep}})$, where $(o_i, o_j) \in \mathcal{E}_{\text{dep}}$ means that o_j cannot be executed before o_i 's result is available. A hardware schedule is a mapping

$$\sigma : \mathcal{O} \rightarrow \mathbb{N}^5, \quad \sigma(o) = (t(o), \text{lane}(o), \text{bank}(o), \text{share}(o), \text{idx}(o)), \quad (4)$$

encoding the time slot, arithmetic lane, memory bank, masking share domain, and canonical transcript index of o . A valid schedule satisfies $(o_i, o_j) \in \mathcal{E}_{\text{dep}} \Rightarrow t(o_i) + \text{latency}(o_i) \leq t(o_j)$. The canonical index $\text{idx}(o)$ is the position to which

Table 1: Factor families in the SDitH v2.0 leakage graph and their leakage relevance.

Family	Algebraic role	Leakage relevance
Regular witness	Each RSD block is a valid unit-vector witness.	Block index determines several downstream tensor and Mux values.
Tensor	Connects e_i to small selectors $u_{i,1}, \dots, u_{i,d}$.	A small selector coordinate can influence many polynomial-evaluation nodes.
Syndrome	Checks $Hx = y$ over $\mathbb{F}_2$.	Matrix-column selection and accumulation leak witness-dependent activity.
Mux	Selects field elements via compressed binary selector bits.	Selector bits gate wide extension-field words; high-SNR control/data interaction.
VOLE correlation	Checks $w_{r,c} = \Delta_c u_{r,c} + v_{r,c}$.	Same correlation value reused across several consistency checks.
BLC	Compresses pseudorandom field elements into accumulators.	Accumulator updates produce repeated, structured leakage over Gray-code order.
Serialization	Restores canonical order before hashing.	Public transcript order helps an attacker align leakage if internal execution order is exposed.

the result of o must eventually be written in the verifier-visible SDitH v2.0 transcript.

In parallel, we model the algebraic part of the signer as a factor graph $\mathcal{G}_{\text{alg}} = (\mathcal{X}, \mathcal{F}, \mathcal{E}_{\text{alg}})$. The variable set $\mathcal{X}$ contains witness bits, tensor selector bits, VOLE-correlation values, BLC accumulator states, extension-field elements, and mask shares. The factor set $\mathcal{F}$ contains the algebraic constraints among these variables; an edge $(X_v, f) \in \mathcal{E}_{\text{alg}}$ exists when variable X_v appears in factor f [33,34]. Table 1 summarizes the main factor families relevant to side-channel inference.

3.2 Leakage-Augmented Factor Graph and SASCA Posterior

For each operation $o \in \mathcal{O}$, let $Z_o = \zeta_o(X_{N(o)})$ denote the intermediate value or transition that influences the physical leakage of o , where $N(o)$ denotes the secret-dependent variables adjacent to o in $\mathcal{G}_{\text{alg}}$. The function ζ_o may represent a Hamming-weight, Hamming-distance, masked share transition, memory-bus value, field-multiplier operand, or Mux control-data interaction. Let Y_o denote the corresponding aligned and preprocessed leakage observation. The leakage likelihood of o is

$$L_o(a) = \Pr[Y_o = y_o \mid Z_o = a]. \quad (5)$$

The leakage-augmented factor graph is $\mathcal{G}_{\text{leak}} = (\mathcal{X} \cup \mathcal{Y}, \mathcal{F} \cup \mathcal{L}, \mathcal{E}_{\text{leak}})$, with $\mathcal{Y} = \{Y_o : o \in \mathcal{O}\}$ and $\mathcal{L} = \{L_o\}$. For a known schedule, the SASCA posterior is

$$\Pr[\mathcal{X} = x \mid \mathbf{Y}, \text{tr}, \sigma] \propto \prod_v P_v(x_v) \prod_{f \in \mathcal{F}} f(x_{N(f)}, \text{tr}) \prod_{o \in \mathcal{O}} L_o(\zeta_o(x_{N(o)})), \quad (6)$$

where tr denotes the public transcript including the message, public key, public seeds, and Fiat–Shamir challenges. The attacker computes posterior beliefs by sum–product message passing or, in the ExSASCA variant, by exact inference over compiled tractable circuits [51,52].

3.3 Hidden Labels and Graph Metrics

In a randomized implementation, the attacker knows the schedule family but not the per-signature schedule seed. The mapping from observations to leakage factors becomes a hidden label $\pi : \{1, \dots, |\mathcal{Y}|\} \rightarrow \mathcal{O}$. Let Π_σ denote the set of assignments consistent with the public schedule family, resource constraints, transcript length, and any synchronization information visible to the attacker. The SASCA posterior generalizes to

$$\Pr[\mathcal{X} = x \mid \mathbf{Y}, \text{tr}] \propto \sum_{\pi \in \Pi_\sigma} \Pr[\pi] \prod_v P_v(x_v) \prod_f f(x_{N(f)}, \text{tr}) \prod_{j=1}^{|\mathcal{Y}|} L_{\pi(j)}(\zeta_{\pi(j)}(x_{N(\pi(j))})). \quad (7)$$

Generic shuffling helps because it increases $|\Pi_\sigma|$, but it may be insufficient: if shuffled operations have distinct leakage shapes, fixed resource locations, or graph-neighborhood signatures, the attacker may recover π before running SASCA [25].

GAVS therefore targets both temporal randomization and graph-aware label ambiguity. Two metrics guide the design. The *secret-reuse degree* $\deg_{\text{sec}}(X_v) = |\{f \in \mathcal{F} : (X_v, f) \in \mathcal{E}_{\text{alg}}\}|$ captures how dangerous a variable is. The *redundancy exposure score* $\text{RES}(\sigma)$ penalizes short-range repeated exposure of high-degree variables through the same lane or memory bank: for $\mathcal{X}_{\text{sec}} \subseteq \mathcal{X}$ the set of attack-relevant variables,

$$\begin{aligned} \text{RES}(\sigma) = \sum_{X_v \in \mathcal{X}_{\text{sec}}} \sum_{\substack{o_i, o_j \in \mathcal{O}(X_v) \\ i < j}} \omega(o_i) \omega(o_j) \exp\left(-\frac{|t(o_i) - t(o_j)|}{\Delta}\right) \\ \cdot (1 + \alpha \mathbf{1}[\text{lane}(o_i) = \text{lane}(o_j)] + \beta \mathbf{1}[\text{bank}(o_i) = \text{bank}(o_j)]), \end{aligned} \quad (8)$$

where $\mathcal{O}(X_v) = \{o : X_v \in N(o)\}$, $\omega(o)$ estimates the leakage strength of o , and α, β, Δ are design constants.

3.4 Threat Model

We consider a passive side-channel attacker with the following capabilities. (T1) The attacker knows the SDitH v2.0 parameter set, signing algorithm, hardware description, scheduling algorithm, and public transcript. (T2) The attacker

measures power or electromagnetic traces aligned at the signature-execution boundary; internal operation labels may be hidden by the randomized schedule. (T3) The attacker may have profiling access to a similar device for training leakage classifiers, or may use non-profiled clustering when profiling access is unavailable [12,7,20,5,8]. (T4) The attacker can run BP-SASCA, exact-inference SASCA [52], deep-learning classification, schedule-label recovery, and permutation-recovery attacks of the doubly-stochastic mix-matrix type used against shuffled NTT implementations [25]. The attacker may also attempt to recover the hidden permutation from canonicalizer-aligned traces; the address-bus reduction of Theorem 6 (Section 6.5) eliminates one channel of this attack by construction, while the control-wire term is bounded empirically by the canonicalizer-focused experiments of Section 8. (T5) The attacker does not know the secret key seed, the witness, the masking randomness, or the per-signature schedule seed ρ_{sch} . (T6) Fault injection, invasive probing, and attacks that modify the verifier-visible protocol are outside the scope of this paper.

4 Attacking an Unprotected SDitH v2.0 Signer

This section instantiates the threat model on a canonical (unprotected) SDitH v2.0 signing implementation. The purpose is to define the baseline attack pipeline against which GAVS is evaluated and to identify the structural features of the deterministic signer that motivate graph-aware scheduling.

4.1 Baseline Signer and Attack Objective

The baseline signer implements the signing path of Section 2.3 under a deterministic schedule σ_0 . Once the parameter set and the hardware description are known, the attacker can map any trace window to its operation label; equivalently, $|H_{\sigma_0}| = 1$. This known-label property allows local leakage likelihoods to be attached directly to the correct factor nodes of the SDitH leakage graph.

The primary attack target is the regular witness $x = e_0 \parallel \dots \parallel e_{w-1}$. Let $p_i \in \{0, \dots, m-1\}$ denote the position of the nonzero entry in block e_i . Recovering $(p_0, \dots, p_{w-1})$ is equivalent to recovering x . The success criteria are full witness recovery $\text{Succ}_{\text{full}}$, block-wise top- k success Succ_k , and average guessing entropy GE, with a candidate validated by checking $\sum_i H[:, im + \hat{p}_i] = y$.

4.2 Attack Surface and Leakage Models

Table 2 summarizes the six leakage targets considered in the baseline attack. These are not independent vulnerabilities but different physical observations of the same structured VOLEitH graph. The Mux-tree combines secret binary selectors with wide field elements; BLC repeatedly updates accumulators in a deterministic Gray-code order. The VOLE consistency equations and the public syndrome equation combine the local evidence into a global posterior.

Table 2: Main leakage targets in the unprotected SDitH v2.0 baseline.

Stage	Target variable and rationale
Witness expansion	Block position p_i and its tensor coordinates determine downstream computations.
Tensor encoding	Selector bits $u_{i,j,\ell}$ influence multiple Mux-tree and polynomial factors.
Mux-tree evaluation	Binary selectors gate $\mathbb{F}_{2^\lambda}$ words; high-amplitude control/data interaction.
VOLE-tag update	Row/column entries reuse algebraically related values across consistency checks.
BLC accumulation	Canonical Gray-code order yields a predictable sequence of accumulator updates.
Hash/serialization	Public synchronization points help align trace windows.

For unmasked registers and buses, leakage is modeled as $Y_o = \alpha_o \text{HW}(Z_o) + \eta_o$ or $Y_o = \alpha_o \text{HD}(Z_o, Z'_o) + \eta_o$ with measurement noise η_o [35]. Local likelihoods $L_o(a)$ are estimated by template attacks [12], correlation power analysis [7], mutual-information analysis [20], or neural classifiers [8, 5]. Extension-field elements over $\mathbb{F}_{2^\lambda}$ with $\lambda \in \{128, 256\}$ are handled either by bit-sliced abstractions (likelihoods on linear projections) or by sparse-candidate abstractions (a small candidate list per local window).

4.3 Known-Label BP-SASCA and Exact-Inference Subgraphs

The principal global attack is BP-SASCA on the leakage-augmented graph of Section 3.2, with σ_0 public. The posterior target factors as

$$\begin{aligned} \Pr[(p_i), U, A, T \mid \mathbf{Y}, \text{tr}, \sigma_0] &\propto F_{\text{RSD}}((p_i), y, H) \cdot F_{\text{tensor}}(U, (p_i)) \cdot F_{\text{Mux}}(U, A, \text{tr}) \\ &\quad \cdot F_{\text{VOLE}}(T, A, \text{tr}) \cdot F_{\text{BLC}}(A, \text{tr}) \cdot \prod_o L_o(\zeta_o(\cdot)), \end{aligned} \tag{9}$$

where U denotes tensor selector variables, A accumulator and polynomial-evaluation variables, and T VOLE-dependent tag variables.

Factor scopes. Each factor in Eq. (9) is an indicator that enforces an algebraic relation among its scope variables: $F_{\text{RSD}}((p_i), y, H) = \mathbf{1}[H \sum_i e_{p_i, i} = y]$, where $e_{p, i}$ is the unit vector at position p in block i ; $F_{\text{tensor}}(U, (p_i))$ is the product over blocks i and dimensions j of $\mathbf{1}[u_{i, j} = \text{tensorize}(p_i)_j]$; $F_{\text{Mux}}(U, A, \text{tr})$ enforces the Mux-tree evaluations of Eq. (2) at every internal node; $F_{\text{VOLE}}(T, A, \text{tr})$ enforces the consistency equations $w_{r, c} = \Delta_c \cdot u_{r, c} + v_{r, c}$ and the row-aggregation $T_r = \sum_c \gamma_c w_{r, c}$; and $F_{\text{BLC}}(A, \text{tr})$ enforces Eq. (3). All factors are public functions of the parameter set; sum-product message passing therefore propagates evidence between the latent witness variables $(p_i), U, A, T$ through the leakage likelihoods $L_o(\zeta_o(\cdot))$ and through these algebraic indicators.

The attack initializes uniform priors on each p_i , injects local evidence, runs sum-product message passing, lists top- K position candidates per block, prunes by the syndrome equation, and reports full-witness recovery on a unique surviving candidate. Local classification errors are corrected by the algebraic factors, which is precisely the failure mode that motivates GAVS.

A loopy-BP failure is not a security boundary. We additionally evaluate ExSASCA [52], which compiles security-critical neighborhoods $\mathcal{G}_i^{\text{Mux}}$, $\mathcal{G}_r^{\text{BLC}}$, $\mathcal{G}_s^{\text{VOLE}}$ into tractable probabilistic circuits and computes sharper posteriors that refine the global BP-SASCA evidence.

4.4 Why Leakage Detection Alone Is Insufficient

Fixed-versus-random and semi-fixed leakage tests following TVLA [22,29] are useful for detecting first-order leakage but are not sufficient for VOLEitH hardware. A design may have leakage below a statistical threshold at individual time samples yet still allow SASCA to aggregate enough weak evidence for full witness recovery. We therefore evaluate the baseline against both leakage-detection metrics and recovery-oriented metrics (Succ_k , $\text{Succ}_{\text{full}}$, GE, and label-recovery accuracy). The baseline is expected to fail four structural conditions: deterministic schedule-to-operation mapping, repeated selector-controlled Mux exposures, predictable BLC traversal, and strong global algebraic constraints that turn local errors into correctable noise. These four failure modes drive the design of GAVS.

5 GAVS Design

GAVS combines the dependency DAG $\mathcal{D}_{\text{hw}}$ with the leakage factor graph $\mathcal{G}_{\text{leak}}$ and an explicit set of resource and security constraints into a single scheduling problem. This section presents the design goals, the scheduling objective, the three randomized scheduling primitives, the label-hiding canonicalization, the masking-aware placement rules, and the seed-handling discipline.

5.1 Design Goals and Scheduler Inputs

GAVS aims to satisfy four goals simultaneously: (G1) the verifier-visible SDitH v2.0 transcript is preserved bit-for-bit; (G2) external timing and resource utilization are constant for fixed parameter sets; (G3) the leakage-to-factor labeling has substantial residual uncertainty, even under exact-inference SASCA; and (G4) the design composes with first- and higher-order Boolean masking through the share-domain assignment $\text{share}(o)$. GAVS is not a replacement for masking; it is a complementary structural defense.

The scheduler operates on five inputs: the dependency DAG $\mathcal{D}_{\text{hw}}$, factor-graph metadata derived offline (operation class $\text{cls}(o)$, observable class $\text{obs}(o)$, secret-neighborhood identifier $\text{sec}(o)$, canonical index $\text{idx}(o)$, resource mask $\text{res}(o)$), the per-stage resource budget $\mathcal{R}$, an independent schedule seed ρ_{sch} , and the

public hash transcript tr . None of the metadata depends on the actual values of secret variables; all metadata refers to public structural classes such as “selector-bit-driven Mux at tree depth h in block i .”

5.2 Scheduling Objective

A valid schedule satisfies all dependency, resource, and share-domain constraints. Among valid schedules, **GAVS** selects one that minimizes

$$J(\sigma) = \lambda_1 \text{RES}(\sigma) + \lambda_2 \text{REP}(\sigma) + \lambda_3 \text{LAB}(\sigma) - \lambda_4 \text{ENT}(\sigma), \quad (10)$$

where $\text{RES}(\sigma)$ is the redundancy-exposure score of Eq. (8), $\text{REP}(\sigma)$ penalizes short-range repetitions of the same operation class on the same lane, $\text{LAB}(\sigma)$ penalizes label-class concentration in a single time window, and $\text{ENT}(\sigma)$ rewards mixing of leakage-similar operations across windows. The weights $\lambda_1, \dots, \lambda_4$ are chosen by design-time calibration against a representative leakage simulator. Because all four terms depend only on σ and on public metadata, $J(\sigma)$ is computable without secret values.

Exact minimization of J is NP-hard. **GAVS** therefore uses a greedy list scheduler with a bounded-lookahead cost engine that evaluates the local marginal cost

$$\begin{aligned} \Delta J(o, t, \ell, b) = & \sum_{o' \in \mathcal{H}_\Delta(t)} \chi(\text{sec}(o), \text{sec}(o')) \exp\left(-\frac{|t-t(o')|}{\Delta}\right) \\ & \cdot (1 + \alpha \mathbf{1}[\ell = \text{lane}(o')] + \beta \mathbf{1}[b = \text{bank}(o')]), \end{aligned} \quad (11)$$

of placing operation o at time t , lane ℓ , and memory bank b . Here $\mathcal{H}_\Delta(t)$ is the set of operations scheduled in the most recent Δ cycles, and $\chi(\cdot, \cdot)$ is a public, family-dependent conflict weight that we instantiate as

$$\chi^{(\text{family})}(\text{sec}(o), \text{sec}(o')) = \sum_{X_v \in \text{sec}(o) \cap \text{sec}(o')} \deg_{\text{sec}}(X_v) \cdot w_{\text{family}}(X_v), \quad (12)$$

where $w_{\text{family}}(X_v) \geq 0$ encodes how dangerous variable X_v is for the corresponding observable class. The VOLE-tag specialization is Eq. (14) below with $w_{\text{VOLE}} \equiv 1$; the Tensor-Mux specialization uses $w_{\text{Mux}}(u_{i,j,\ell}) = \gamma_3$ if X_v is a selector bit, γ_2 if it is a tensor coordinate, and γ_1 if it is a block index, with $\gamma_1 \leq \gamma_2 \leq \gamma_3$. A constant-size recent-history table indexed by a hash of $\text{sec}(o)$ enables $O(1)$ evaluation of Eq. (11), keeping the scheduler critical path bounded.

5.3 Randomized Hypercube-Automorphic Gray Traversal for BLC

Let $N = 2^\kappa$ and let $g_j = j \oplus (j \gg 1)$ be the standard binary reflected Gray code on $\{0, 1\}^\kappa$. From $\rho_{\text{BLC}} = \text{PRG}(\rho_{\text{sch}}, \text{“BLC”})$, **GAVS** samples a coordinate permutation $P_\rho \in S_\kappa$, a translation vector $b_\rho \in \{0, 1\}^\kappa$, a rotation offset $r_\rho \in \mathbb{Z}_N$, and a direction bit $d_\rho \in \{+1, -1\}$. The randomized traversal sequence is

$$p_j = P_\rho(g_{(r_\rho + d_\rho j) \bmod N}) \oplus b_\rho, \quad j = 0, 1, \dots, N-1. \quad (13)$$

Coordinate permutation, translation, rotation, and reversal are all automorphisms of the κ -dimensional hypercube; their composition is itself an automorphism. Theorems 1 and 2 in Section 6 establish that Eq. (13) is a Hamiltonian cycle on $\{0,1\}^\kappa$ with one-bit transitions and that both BLC accumulators of Eq. (3) are invariant under this randomization.

5.4 Two-Dimensional VOLE-Tag Scheduler

The VOLE-tag computation is naturally indexed by row and column, $\mathcal{O}_{\text{VOLE}} = \{o_{r,c} : 0 \leq r < R, 0 \leq c < C\}$, with $w_{r,c} = \Delta_c \cdot u_{r,c} + v_{r,c}$ over $\mathbb{F}_{2^\lambda}$. The scheduler processes VOLE operations in fixed-size tiles

$$\mathcal{T}_{a,b} = \{(r, c) : aW_R \leq r < (a+1)W_R, bW_C \leq c < (b+1)W_C\},$$

where W_R, W_C are public parameters. Tiles are padded with dummy operations whose external resource footprint matches that of real operations, so every tile has identical observable cost.

Within each tile, the per-signature order is chosen using $\rho_{\text{VOLE}} = \text{PRG}(\rho_{\text{sch}}, \text{"VOLE"})$ and the VOLE conflict score

$$\chi_{\text{VOLE}}((r, c), (r', c')) = \sum_{X_v \in \text{sec}(o_{r,c}) \cap \text{sec}(o_{r',c'})} \deg_{\text{sec}}(X_v), \quad (14)$$

which is high precisely when two operations expose the same VOLE-dependent or witness-dependent linear form. The local placement cost is Eq. (11) with χ instantiated to χ_{VOLE} . Three effects follow: repeated exposures of the same VOLE tag are separated in time; when separation is impossible, different lanes and banks are chosen; and operations with the same observable class but different secret neighborhoods are mixed within each tile to increase label ambiguity. Section 6 establishes value invariance, constant-work execution, and masking-composability for this scheduler.

5.5 Tensor-Mux Scheduler

A selector bit $u_{i,j,\ell}$ with $\ell \geq 1$ may control several Mux-tree nodes derived from the same witness block i . The Mux-tree conflict weight is the specialization of Eq. (12) with w_{Mux} as defined in Section 5.2: γ_3 for shared selector bits, γ_2 for shared tensor coordinates, γ_1 for the same block. The Tensor-Mux scheduler applies three rules using $\rho_{\text{Mux}} = \text{PRG}(\rho_{\text{sch}}, \text{"MUX"})$.

Block interleaving. Instead of completing the full Mux tree of block i before moving to block $i+1$, the scheduler interleaves blocks under a per-signature permutation $\pi_{\rho_{\text{Mux}}}(0, 1, \dots, w-1)$ over a public padded block window. No single trace segment contains a dense cluster of leakage about one witness block.

Level separation. For two Mux-tree operations $o_{i,h}$ and $o_{i,h'}$ from the same block but different tree levels, the soft constraint $|t(o_{i,h}) - t(o_{i,h'})| \geq D_{\text{level}}$ is enforced whenever dependency and resource constraints allow. The lower bound

D_{level} is set to at least the masked-gadget refresh latency τ_{refresh} in masked builds.

Lane rotation. If two operations share a selector coordinate $u_{i,j,\ell}$ with $\ell \geq 1$, they are assigned to distinct lanes whenever possible:

$$u_{i,j,\ell} \in \text{sec}(o_a) \cap \text{sec}(o_b), \ell \geq 1 \Rightarrow \text{lane}(o_a) \neq \text{lane}(o_b), \quad (15)$$

otherwise the later operation is delayed by one cycle or paired with a balanced dummy slot. None of these rules alters the tensor-product relation; they only change the order and placement of independent Mux-tree operations.

5.6 Label-Hiding Canonicalization

A randomized internal order is useful only if the canonicalization stage does not reveal the entire schedule through high-SNR addresses or control signals. GAVS separates randomized result production from public transcript serialization through a fixed-topology Beneš network whose routing-control bits are computed from *public canonical indices only*.

During each macro-window, real and dummy results are written to scratch in time order, $s = 0, 1, \dots, M-1$, so the scratch write address is independent of the canonical index. Each real result carries an internal tag $\text{idx}(o)$ that remains inside the canonicalization unit. At the end of the macro-window, the canonicalizer applies the inverse permutation of the spatial tag arrangement, restoring canonical order on a fixed-scan write-back path. The canonicalization invariant is

$$\text{Serialize}_{\text{GAVS}}(\{(\text{idx}(o), \text{value}(o))\}_{o \in \mathcal{O}_{\text{out}}}) = \text{Serialize}_{\text{SDitH}}(\{\text{value}(o)\}_{o \in \mathcal{O}_{\text{out}}}). \quad (16)$$

Section 7.6 specifies the hardware construction, and Theorem 6 in Section 6.5 establishes that the canonicalizer’s external address bus is independent of ρ_{sch} by construction and that the residual control-wire term is bounded empirically.

5.7 Schedule Seeds, Dummy Operations, and Masking-Aware Placement

The schedule seed is derived independently of all secret-dependent values through domain separation:

$$\begin{aligned} \rho_{\text{BLC}} &= \text{PRG}(\rho_{\text{sch}}, \text{“BLC”}), & \rho_{\text{VOLE}} &= \text{PRG}(\rho_{\text{sch}}, \text{“VOLE”}), \\ \rho_{\text{Mux}} &= \text{PRG}(\rho_{\text{sch}}, \text{“MUX”}), & \rho_{\text{dummy}} &= \text{PRG}(\rho_{\text{sch}}, \text{“DUMMY”}). \end{aligned} \quad (17)$$

The PRG used for schedule expansion is logically distinct from the PRG used by the SDitH proof system; this prevents coupling between protocol randomness and hiding randomness. To prevent leakage of ρ_{sch} through arithmetic side channels in the schedule-sampling logic, all run-time permutation generation uses power-of-two domain sizes so that modular reduction reduces to bit masking,

and rejection sampling with a public ceiling is used whenever non-power-of-two ranges are unavoidable [44].

Dummy operations keep the cycle envelope constant in two situations: when dependency constraints leave empty slots, and when operation-class counts within a scheduling window must be made indistinguishable. A dummy operation has the same public class, resource footprint, and memory-access pattern as a real operation in the same macro-stage; its result is discarded before canonical serialization.

For masking-aware placement, let a sensitive variable X be represented by $d_{\text{mask}} + 1$ Boolean shares $X = X^{(0)} \oplus \dots \oplus X^{(d_{\text{mask}})}$. GAVS requires that shares of the same logical value processed in the same masked gadget instance occupy distinct share domains:

$$\text{share}(o^{(a)}) \neq \text{share}(o^{(b)}) \quad \text{for all } a \neq b, \quad (18)$$

together with lane rotation across shares whenever the gadget requires physical lane separation, and level separation $D_{\text{level}} \geq \tau_{\text{refresh}}$ between consecutive masked operations on the same logical variable.

6 Security Analysis

This section develops the formal security analysis of GAVS. We establish transcript equivalence, constant-work execution, and a three-layer composability stack with masking covering Probe-Isolating Non-Interference (PINI) preservation, glitch- and transition-extended robust probing, and physical share-domain segregation. We also derive an M -class label-entropy bound from the Audenaert–Fannes continuity inequality together with a Le Cam Top-1 argument, complemented by a min-entropy corollary aligned with the operational side-channel security metric. All claims are proved in full.

6.1 Scope and Assumptions

The security claims of this section concern the implementation of SDitH v2.0 signing in GAVS-style hardware. We do not modify the SDitH proof system, public transcript distribution, or Fiat–Shamir security. The formal package rests on a layered set of assumptions matched to layered security properties.

(A1) Abstract PINI.

Every primitive masked gadget (field multiplication, field addition, masked Mux, masked refresh) is t -PINI in the abstract probing model of [10].

(A1') Glitch- and transition-extended PINI.

Every primitive masked gadget is additionally t -PINI in the glitch- and iterated-transition-extended robust probing model [18,11]. Concretely, the gadget library is realized through HPC2 with explicit output refresh [9], HPC3 [31,50], or an O-PINI2 variant [11].

(A1'') Physical share-domain segregation.

The hardware datapath statically partitions scratch memory and arithmetic lanes across share domains: physical resources used to carry share i of any logical value are disjoint from those used to carry share j for $j \neq i$ [16,14].

(A2) BLC operand stream binding.

The BLC operand stream is generated by a public function $u \mapsto r_u$ of the canonical domain point, independent of the per-signature traversal order.

(A3) Schedule-seed independence.

The schedule seed ρ_{sch} is sampled from a true random source or a cryptographically strong PRG independently of secret-dependent values.

(A4) Canonical-index tagging.

Every result operation o carries its canonical index $\text{idx}(o)$ alongside its value through scratch storage.

Assumptions (A1) and (A1') are discharged by the choice of gadget library and by formal masking-verification tools [2,46,45,32]. Assumption (A1'') is discharged by floorplan-level placement constraints and the RTL assertions of Section 7.5. Each layer is independently verifiable, and a failure at one layer does not invalidate the results at the others.

6.2 Transcript Equivalence

We begin with the BLC primitive and then extend the equivalence to Tensor-Mux and VOLE-Tag.

Theorem 1 (BLC traversal correctness). *Let $N = 2^\kappa$, let $g_j = j \oplus (j \gg 1)$ be the standard binary reflected Gray code on $\{0, 1\}^\kappa$, and let $P_\rho \in S_\kappa$, $b_\rho \in \{0, 1\}^\kappa$, $r_\rho \in \mathbb{Z}_N$, $d_\rho \in \{+1, -1\}$ be sampled from ρ_{BLC} . Define $p_j = P_\rho(g_{(r_\rho + d_\rho j) \bmod N}) \oplus b_\rho$ for $j = 0, \dots, N-1$. Then $\{p_0, \dots, p_{N-1}\} = \{0, 1\}^\kappa$ and $\text{wt}(p_{j+1} \oplus p_j) = 1$ for every j (indices modulo N).*

Proof. We prove (i) the sequence is a permutation of $\{0, 1\}^\kappa$ and (ii) consecutive elements differ in exactly one coordinate.

Part (i). The map $j \mapsto (r_\rho + d_\rho j) \bmod N$ is a bijection $\mathbb{Z}_N \rightarrow \mathbb{Z}_N$: it is a composition of an additive shift $j \mapsto j + r_\rho$ with either the identity ($d_\rho = +1$) or the reflection $j \mapsto -j$ ($d_\rho = -1$), each of which is a bijection on $\mathbb{Z}_N$. The standard BRGC map $i \mapsto g_i$ is a bijection $\mathbb{Z}_N \rightarrow \{0, 1\}^\kappa$. Coordinate permutation by $P_\rho \in S_\kappa$ is a bijection $\{0, 1\}^\kappa \rightarrow \{0, 1\}^\kappa$. Translation $v \mapsto v \oplus b_\rho$ is its own inverse and thus a bijection. The composition of bijections is a bijection, so $j \mapsto p_j$ is a bijection from $\{0, \dots, N-1\}$ onto $\{0, 1\}^\kappa$.

Part (ii). Write $j' = (r_\rho + d_\rho j) \bmod N$ and $j'' = (r_\rho + d_\rho(j+1)) \bmod N$, so $j'' \equiv j' + d_\rho \pmod{N}$. Then

$$p_{j+1} \oplus p_j = (P_\rho(g_{j''}) \oplus b_\rho) \oplus (P_\rho(g_{j'}) \oplus b_\rho) = P_\rho(g_{j''}) \oplus P_\rho(g_{j'}) = P_\rho(g_{j''} \oplus g_{j'}),$$

where the last equality uses the $\mathbb{F}_2$ -linearity of the coordinate-permutation action. If $d_\rho = +1$, the one-bit-transition property of BRGC gives $\text{wt}(g_{j'+1 \bmod N} \oplus$

$g_{j'}) = 1$; for $d_\rho = -1$, the same property applies symmetrically because BRGC is a Hamiltonian cycle. Coordinate permutation preserves Hamming weight, so $\text{wt}(p_{j+1} \oplus p_j) = \text{wt}(g_{j''} \oplus g_{j'}) = 1$. $\square$

Theorem 2 (BLC value preservation). *Under assumption (A2), let $r_u \in \mathbb{F}_{2^\lambda}$ be associated with canonical domain point $u \in \{0, 1\}^\kappa$ as a public function of u . Let $p_0, \dots, p_{N-1}$ be the randomized traversal of Eq. (13). Then both BLC accumulators of Eq. (3) are invariant:*

$$\sum_{j=0}^{N-1} r_{p_j} = \sum_{u \in \{0, 1\}^\kappa} r_u, \quad \sum_{j=0}^{N-1} p_j \cdot r_{p_j} = \sum_{u \in \{0, 1\}^\kappa} u \cdot r_u.$$

Proof. By Theorem 1, $\{p_0, \dots, p_{N-1}\}$ equals $\{0, 1\}^\kappa$ as a set. By assumption (A2), r_{p_j} depends only on the canonical point p_j , not on the traversal index j ; therefore $r_{p_j} = r_u$ whenever $p_j = u$. Re-indexing the sum from the traversal index j to the canonical point $u = p_j$ is valid because addition in $(\mathbb{F}_{2^\lambda}, +)$ is associative and commutative. The same re-indexing applies coordinate-wise to the second accumulator in $(\mathbb{F}_{2^\lambda}^\kappa, +)$, yielding the canonical value $\sum_u u \cdot r_u$. $\square$

Theorem 3 (Tensor-Mux output equivalence). *Fix the SDitH v2.0 parameter set, the public message, the secret witness, the protocol signing randomness, and the schedule seed ρ_{Mux} . Let σ_{Mux} be the per-signature schedule produced by the Tensor-Mux scheduler of Section 5.5 subject to the dependency constraints of $\mathcal{D}_{\text{hw}}$, the canonical-index tagging contract, and the masking-aware placement of Eq. (18). Then for every Mux instance, the extension-field value emitted to scratch under σ_{Mux} equals the value emitted under the canonical schedule σ_0 .*

Proof. We proceed by structural induction on the Mux tree levels of a single block i . The cross-block claim follows because block evaluations are algebraically independent in the SDitH v2.0 polynomial interactive oracle proof.

Base case (level 0, tensor leaves). A level-0 Mux node o has inputs $z_0, \dots, z_{\mu-1} \in \mathbb{F}_{2^\lambda}$ (field constants from public seed expansion) and selector bits $b_1, \dots, b_{\mu-1} \in \mathbb{F}_2$ (from the compressed tensor witness of Eq. (1)), and computes $z_{\text{out}}(o) = z_0 + \sum_{\ell=1}^{\mu-1} b_\ell(z_\ell + z_0)$. The seed-expansion engine produces z_ℓ deterministically from the public seed; the scheduler does not modify this stage. The witness-expansion engine produces b_ℓ deterministically from the secret witness; the scheduler does not modify this engine. Therefore the inputs to o under σ_{Mux} are bit-identical to the inputs under σ_0 . Since the Mux polynomial is a deterministic function of its inputs, $z_{\text{out}}(o)|_{\sigma_{\text{Mux}}} = z_{\text{out}}(o)|_{\sigma_0}$.

Inductive step. Assume every Mux output at levels $< h$ within block i matches σ_0 . Consider a level- h node o . Its inputs are: (a) the outputs of μ level- $(h-1)$ Mux nodes, which by the inductive hypothesis equal their σ_0 values; and (b) compressed selector bits from the witness, unchanged by scheduling. The dependency DAG $\mathcal{D}_{\text{hw}}$ forces the ready-queue logic to issue o only after all level- $(h-1)$ predecessors complete; hence o consumes the same field inputs and the same selector bits as it would under σ_0 . The Mux polynomial is a fixed

polynomial over $\mathbb{F}_{2^\lambda}$, depending only on inputs and not on cycle, lane, memory bank, or share-domain location. The XOR-network implementation may sum the addends $b_\ell(z_\ell + z_0)$ in any order, but $(\mathbb{F}_{2^\lambda}, +)$ is associative and commutative, so any summation order yields the same field value. Therefore $z_{\text{out}}(o)|_{\sigma_{\text{Mux}}} = z_{\text{out}}(o)|_{\sigma_0}$.

By induction, every Mux output at every level of every block matches the canonical value. The canonical-index tagging contract ensures each output carries $\text{idx}(o)$ into scratch; the label-hiding canonicalizer of Section 5.6 re-emits scratch contents in canonical idx order. The multiset of (canonical-index, value) pairs delivered to the hash engine under σ_{Mux} is therefore identical, in canonical order, to that produced under σ_0 . $\square$

Theorem 4 (VOLE-Tag output equivalence). *Let σ_{VOLE} be the per-signature schedule produced by the two-dimensional VOLE-tag scheduler of Section 5.4 under fixed signing randomness, secret witness, public transcript, and schedule seed ρ_{VOLE} . Then every VOLE-tag accumulator value and every consistency-check value emitted to the hash boundary under σ_{VOLE} equals the canonical value under σ_0 .*

Proof. We decompose the VOLE-tag computation into two algebraic levels.

Level 1 (per-entry). Within tile $\mathcal{T}_{a,b}$, each entry $w_{r,c} = \Delta_c \cdot u_{r,c} + v_{r,c}$ depends on three inputs over $\mathbb{F}_{2^\lambda}$: the column challenge Δ_c (a public Fiat–Shamir value), the witness-derived value $u_{r,c}$ (from the witness-expansion engine, which the scheduler does not modify), and the public PRG output $v_{r,c}$ keyed to the canonical (r, c) coordinate. None depends on intra-tile scheduling order. Therefore $w_{r,c}|_{\sigma_{\text{VOLE}}} = w_{r,c}|_{\sigma_0}$ for every (r, c) .

Level 2 (cross-column aggregation). The tag-row consistency check forms $T_r = \sum_{c=0}^{C-1} \gamma_c \cdot w_{r,c}$ with public combinators γ_c . By Level 1, each summand has the canonical value. The sum is over the abelian group $(\mathbb{F}_{2^\lambda}, +)$, so any permutation of summation order yields the same T_r .

The scheduler may permute the order in which (r, c) pairs are computed within and across tiles, but the row-aggregation summation order is determined by the canonicalization stage, not by σ_{VOLE} . The fixed-scan canonicalizer reads tagged entries from scratch in canonical (r, c) order, so the hash engine performs the aggregation in the same canonical order under both schedules. Hence the verifier-visible transcript, consisting of $\{T_r\}_r$ and any Fiat–Shamir-revealed subset of $\{w_{r,c}\}$, is bit-identical between σ_{VOLE} and σ_0 . $\square$

Theorem 5 (Stack-wide transcript equivalence). *Fix the SDitH v2.0 parameter set, public message, secret key, protocol signing randomness, and schedule seed ρ_{sch} . If Theorems 1, 2, 3, 4 all hold and the canonicalization buffer satisfies Eq. (16), then the verifier-visible signature produced by GAVS is bit-identical to the canonical SDitH v2.0 signature.*

Proof. The SDitH v2.0 signing path comprises six macro-stages. GAVS modifies the execution order within each macro-stage; it does not modify the hash-absorption order, which is a public sequence emitted by the canonicalizer. By

Theorems 1 and 2, every BLC accumulator value in the commitment and VOLE-generation stage equals its canonical counterpart. By Theorem 4, every VOLE-tag and row-consistency value in that same stage matches σ_0 . By Theorem 3, every Mux-tree polynomial-check output matches σ_0 . Seed expansion and witness expansion are not reordered. Challenge derivation hashes the canonically ordered transcript prefix, which is bit-identical to that of σ_0 by the previous statements; hence the Fiat–Shamir challenge values are identical. The final signature serialization, performed by the canonicalizer, emits the same canonical byte sequence under both schedules. $\square$

Corollary 1 (Cryptographic preservation). *Under the conditions of Theorem 5, the verifier-visible signature distribution of GAVS equals that of canonical SDitH v2.0. Any existential forgery attack on the GAVS signature distribution implies an attack on canonical SDitH v2.0 with the same protocol randomness.*

6.3 Constant-Work Execution

Proposition 1 (Constant-work execution). *For a fixed SDitH v2.0 parameter set and public message length, the sequence of macro-stage lengths, the number of real and dummy slots, the number of memory transactions, the number of arithmetic-lane activations, and the number of canonicalization-buffer accesses are functions only of the public parameters and ρ_{sch} . None of these depends on the secret witness.*

Proof. The metadata fields $\text{cls}(o)$, $\text{obs}(o)$, $\text{sec}(o)$, $\text{res}(o)$, and $\text{idx}(o)$ are determined by the public algorithmic structure of SDitH v2.0, not by actual secret values. The scheduling objective $J(\sigma)$ of Eq. (10) and the local cost ΔJ of Eq. (11) depend on these public metadata fields and on ρ_{sch} , which by assumption (A3) is independent of the secret. Each macro-stage is padded to a public size, and dummy operations fill empty slots with the same public class and resource footprint as real operations. Therefore the macro-stage envelope and per-stage resource ledger are public functions of the parameter set, public message length, architecture, and ρ_{sch} , but not of the secret witness. $\square$

Proposition 2 (Secret-independent memory access pattern). *Let $\mathcal{M}_{\rho_{\text{sch}}}$ be the ordered list of memory access types generated by GAVS for one signing execution, recording for each access whether it is a read or write, the public macro-stage, the memory bank, and whether the access targets scratch or canonicalization storage. Then $\Pr[\mathcal{M}_{\rho_{\text{sch}}} = m \mid x] = \Pr[\mathcal{M}_{\rho_{\text{sch}}} = m \mid x']$ for any two valid witnesses x, x' under fixed public parameters and message length.*

Proof. The distribution of $\mathcal{M}_{\rho_{\text{sch}}}$ is induced by ρ_{sch} , public metadata, and fixed padding rules. Since ρ_{sch} is independent of the secret witness by assumption (A3) and the scheduler does not branch on secret-dependent data (Proposition 1), the two conditional distributions are identical. $\square$

6.4 Composability with Masking

GAVS and masking address different parts of the side-channel problem. Masking reduces the information contained in local leakage; GAVS reduces the attacker’s ability to align and combine many local leakages through the VOLEitH factor graph. Composing the two correctly requires care, because the literature distinguishes three logically separate security properties:

(L1) Probe-Isolating Non-Interference.

A composability notion for masked gadgets, defined purely in terms of share indices [10]. Trivial composition of t -PINI gadgets, with share-index disjointness preserved at every interface, yields a t -PINI composite.

(L2) Glitch- and transition-extended robust probing.

An extension of the probing model that admits physical defaults such as glitches and transitions on combinational and memory cells [18,11]. Modern hardware gadget families such as HPC2, HPC3, and O-PINI2 are designed to be t -PINI in this stronger model [9,31,50].

(L3) Physical share-domain segregation.

A placement and routing discipline that prevents distinct shares of the same logical value from sharing physical resources—a register, a wire bundle, or a memory bank—in temporally adjacent windows [16,14].

We state one lemma per layer for the Tensor-Mux engine, then mirror the structure for the VOLE-Tag engine. Combined corollaries follow.

Lemma 1 (Tensor-Mux PINI preservation). *Under assumption (A1), if the Tensor-Mux scheduler enforces share-domain disjointness at every gadget interface—that is, every gadget instance receives input shares matched to its share-index slot and writes its output shares to share-indexed scratch locations—then the composed Tensor-Mux circuit is t -PINI.*

Proof. Every component gadget is t -PINI by (A1). The composed circuit is a DAG of these gadgets and their refresh gadgets, with inter-gadget interfaces consisting of scratch reads and writes that carry one share per wire. The scheduling discipline sends share- i operations to share- i -tagged scratch locations and never crosses share indices, so every interface satisfies the share-index disjointness hypothesis of the trivial-composition theorem of [10, Theorem 3]. The theorem yields t -PINI for the composite. $\square$

Lemma 2 (Tensor-Mux robust-probing implementability). *Under assumption (A1’), if the Tensor-Mux scheduler enforces level separation $D_{\text{level}} \geq \tau_{\text{refresh}}$ between any two same-logical-variable masked operations at different Mux-tree depths in the same block, and the gadget library realizes each Mux gadget through register-bounded combinational stages between adjacent levels, then the composed Tensor-Mux circuit is t -PINI in the glitch-extended and iterated-transition-extended robust probing model.*

Proof. By (A1'), every component Mux gadget is t -PINI in the combined glitch- and iterated-transition-extended probing model. The register boundary blocks glitch propagation across Mux-tree levels [18, Section 3]. The level-separation constraint $D_{\text{level}} \geq \tau_{\text{refresh}}$ ensures that consecutive uses of the same logical variable are separated by at least one refresh latency, preventing iterated-transition probes from coupling those uses [11, Theorem 4]. Trivial composition for glitch-robust PINI [9, Theorem 1] then gives the conclusion. $\square$

Lemma 3 (Tensor-Mux physical share-domain segregation). *Under assumption (A1''), if the Tensor-Mux scheduler enforces $\text{lane}(o_a) \neq \text{lane}(o_b)$ whenever $\text{share}(o_a) \neq \text{share}(o_b)$ and o_a, o_b depend on the same logical variable, with delayed-or-dummy fallback when forced to the same lane by resource exhaustion, then no physical lane or bank carries values derived from two different share domains of the same logical variable within any window of $\leq D_{\text{level}}$ cycles.*

Proof. The static partitioning of (A1'') eliminates concurrent share-domain collisions on lanes and banks. The lane-rotation rule of Eq. (15) eliminates sequential collisions on the same lane within a D_{level} -cycle window. The delayed-or-dummy fallback (Section 5.5) preserves the segregation property under resource exhaustion. Consequently, the physical-leakage observable at any single lane or bank, integrated over any window of $\leq D_{\text{level}}$ cycles, is a function of values from at most one share domain [16,14]. $\square$

Corollary 2 (Combined Tensor-Mux security). *Under (A1), (A1'), (A1'') and the scheduling conditions of Lemmas 1, 2, 3, the composed Tensor-Mux circuit is t -PINI in the glitch- and transition-extended robust probing model, with no first-order leakage from physical share-domain collision on shared lanes or banks.*

The VOLE-Tag engine is treated by three lemmas of identical structure, with the Mux-tree algebra replaced by $w_{r,c} = \Delta_c \cdot u_{r,c} + v_{r,c}$ and the same-logical-variable relation generated by the column-shared Δ_c .

Lemma 4 (VOLE-Tag PINI preservation). *Under (A1), if the VOLE-tag scheduler enforces share-domain disjointness at every masked multiplier and adder interface, then the composed VOLE-tag circuit is t -PINI.*

Proof. The argument is the same as the proof of Lemma 1, with the Mux-tree DAG replaced by the row/column VOLE tile DAG. Each masked multiplier and adder receives input shares matched to its share-index slot and writes output shares to share-indexed scratch locations; the trivial-composition theorem of [10, Theorem 3] then yields t -PINI. $\square$

Lemma 5 (VOLE-Tag robust-probing implementability). *Under (A1'), if the VOLE-tag scheduler enforces the row-uniformity reject-and-resample condition of Section 5.4 and the masked multiplier explicitly refreshes the column-shared Δ_c between consecutive uses, then the composed VOLE-tag circuit is t -PINI in the glitch-extended and iterated-transition-extended robust probing model.*

Proof. The argument mirrors that of Lemma 2. Every masked multiplier and adder is t -PINI in the combined glitch- and iterated-transition-extended probing model by (A1'). Register boundaries between consecutive masked operations block glitch propagation [18, Section 3]. The row-uniformity reject-and-resample step rejects greedy placements that would violate probe-set isolation, ensuring that the placement that is actually executed satisfies the trivial-composition hypothesis. Explicit refresh of Δ_c between consecutive multiplications addresses the column-shared reuse that would otherwise enable iterated-transition probes [11, Theorem 4]. The trivial-composition theorem for glitch-robust PINI [9, Theorem 1] then yields the result. $\square$

Lemma 6 (VOLE-Tag physical share-domain segregation). *Under (A1''), if the VOLE-tag scheduler enforces lane rotation across shares of any masked Δ_c , $u_{r,c}$, $v_{r,c}$, or $w_{r,c}$, then no physical lane or bank carries values from two different share domains of the same logical variable within any window of $\leq D_{\text{level}}$ cycles.*

Proof. The argument mirrors that of Lemma 3. Static partitioning under (A1'') eliminates concurrent share-domain collisions; the lane-rotation rule of Section 5.4 eliminates sequential collisions within a D_{level} -cycle window; resource-exhaustion fallback by delay or dummy preserves segregation. The physical-leakage observable at any single lane or bank therefore depends on values from at most one share domain. $\square$

Corollary 3 (Combined VOLE-Tag security). *Under (A1), (A1'), (A1'') and the scheduling conditions of Lemmas 4, 5, 6, the composed VOLE-tag circuit is t -PINI in the glitch- and transition-extended robust probing model, with no first-order leakage from physical share-domain collision on shared lanes or banks.*

6.5 Security of the Canonicalization Boundary

The canonicalization buffer must not allow the attacker to recover the per-signature schedule from external observations. Recent results have shown that even uniformly random shuffles can be inverted from side-channel observations when an aggregator factor graph couples observation samples to permutation hypotheses [25,23]; this motivates a precise treatment.

Definition 1 (Canonicalizer mutual-information invariant). *Let Π_σ denote the per-signature schedule random variable, and let Y_{canon} denote the multi-dimensional leakage observation aligned to the canonicalization macro-stage. The canonicalizer satisfies the mutual-information invariant if*

$$I(\Pi_\sigma; Y_{\text{canon}} \mid \text{tr, RTL}) \leq \varepsilon_{\text{canon}}$$

for some design parameter $\varepsilon_{\text{canon}}$ bounded by the measurement noise floor of the target platform.

Lemma 7 (Decomposition). *Decompose $Y_{\text{canon}} = (Y_{\text{ctrl}}, Y_{\text{addr}}, Y_{\text{data}})$ into internal control-signal leakage, external address-bus leakage, and external data-bus leakage. Then*

$$\begin{aligned} I(\Pi_\sigma; Y_{\text{canon}} \mid \text{tr}, \text{RTL}) &= I(\Pi_\sigma; Y_{\text{ctrl}} \mid \text{tr}, \text{RTL}) \\ &\quad + I(\Pi_\sigma; Y_{\text{addr}} \mid Y_{\text{ctrl}}, \text{tr}, \text{RTL}) \\ &\quad + I(\Pi_\sigma; Y_{\text{data}} \mid Y_{\text{ctrl}}, Y_{\text{addr}}, \text{tr}, \text{RTL}). \end{aligned}$$

Proof. This is the chain rule for mutual information applied to $X = \Pi_\sigma$ and $(Y_1, Y_2, Y_3) = (Y_{\text{ctrl}}, Y_{\text{addr}}, Y_{\text{data}})$, with every term additionally conditioned on (tr, RTL) . The identity $I(X; Y_1, Y_2, Y_3 \mid C) = I(X; Y_1 \mid C) + I(X; Y_2 \mid Y_1, C) + I(X; Y_3 \mid Y_1, Y_2, C)$ holds by definition of conditional mutual information [15]. $\square$

Theorem 6 (Public-Index Reduction). *If the canonicalizer is implemented as the Beneš construction of Section 7.6 with a fixed-scan write-back FSM, and the control words are zeroized at the end of each macro-window, then:*

1. $I(\Pi_\sigma; Y_{\text{addr}} \mid \text{tr}, \text{RTL}) = 0$ unconditionally.
2. $I(\Pi_\sigma; Y_{\text{ctrl}} \mid \text{tr}, \text{RTL}) \leq \varepsilon_{\text{canon}}$, where $\varepsilon_{\text{canon}}$ is the platform-dependent residual switching-activity leakage of the internal control wires, measured empirically by experiments E4 and E5 of Section 8.3.

Proof. Part (1). The fixed-scan write-back FSM emits the sequence $0, 1, \dots, N-1$ of canonical indices on the external address bus, regardless of which scratch slot holds which canonical value. This sequence is a deterministic function of N , which is public and fixed by the parameter set. Therefore Y_{addr} is a function of RTL alone, independent of Π_σ conditionally on (tr, RTL) , and the conditional mutual information vanishes.

Part (2). The routing-control bits are computed by the standard algorithm of [37] applied to the inverse of the scratch-tag permutation π , where $\pi(s) = \text{idx}(\text{value in slot } s)$. The multiset of input tags is always $\{0, 1, \dots, N-1\}$ by the canonical-index tagging contract (A4), but their *spatial arrangement* in scratch is determined by ρ_{sch} via the upstream scheduler. Hence the routing-control bit values are schedule-dependent. The zeroization discipline ensures that no control word persists across macro-window boundaries and no control signal traces a combinational path to any external pin. The only externally observable manifestation of Y_{ctrl} is then residual switching-activity leakage on internal control wires, which we denote $\varepsilon_{\text{canon}}$. By construction this quantity is bounded by the platform’s measurement noise floor; we measure it empirically through E4 (schedule-label recovery) and E5 (schedule-seed distinguishability) on canonicalizer-aligned traces. $\square$

Corollary 4 (Combined stack security). *Under (A1), (A1’), (A1’’) and the share-disjoint storage discipline of Eq. (18), the composed GAVS-M signing accelerator (BLC datapath, Tensor-Mux engine, VOLE-tag engine, canonicalizer, hash absorption) satisfies the following three properties:*

1. *t-PINI in the abstract probing model, by trivial composition of Lemmas 1 and 4 with the canonicalizer property of Theorem 6 and the share-by-share linearity of the BLC datapath [10, Theorem 3].*
2. *t-PINI in the glitch- and iterated-transition-extended robust probing model, by trivial composition of Lemmas 2 and 5 with the register-bounded canonicalizer construction of Section 7.6.*
3. *Free of first-order leakage from physical share-domain collision on shared lanes or banks, by composition of Lemmas 3 and 6 with the static bank partitioning of Section 7.5.*

Proof. Part (1) follows from the three component PINI lemmas and the trivial-composition theorem of [10, Theorem 3]. Inter-component interfaces are scratch reads and writes that carry one share per wire, preserving share-index disjointness. Part (2) follows from the three robust-probing lemmas and the trivial-composition theorem of [9, Theorem 1]; the canonicalizer’s register-bounded Beneš stages do not violate share-disjointness because each share traverses the network through identical public-index control bits (Theorem 6). Part (3) follows from Lemmas 3 and 6 together with the static partitioning of scratch banks across share domains. $\square$

6.6 Label-Entropy Bound

We bound a classifier’s ability to identify the hidden label of a leakage window in a quantity that is measurable on hardware. Let I be a scheduling window and q an observable leakage class. Define

$$S_{I,q}(\sigma) = \{o \in \mathcal{O} : t(o) \in I, \text{ obs}(o) = q\}, \quad M_{I,q} = |S_{I,q}(\sigma)|,$$

and let $\Pi_{I,q} \in S_{I,q}(\sigma)$ be the random variable that selects the true label of a leakage observation in this window.

Theorem 7 (M -class label-entropy bound). *Let Π be uniformly distributed over a set S of size $M \geq 2$, let $\mathbf{Y}$ be the leakage observation aligned to the corresponding window, and let $P_{\mathbf{Y}|\pi}$ denote the leakage distribution conditioned on $\Pi = \pi$. Define*

$$\varepsilon = \max_{\pi, \pi' \in S} \text{TV}(P_{\mathbf{Y}|\pi}, P_{\mathbf{Y}|\pi'}).$$

Then any classifier $\hat{\Pi} = f(\mathbf{Y})$ satisfies

$$\Pr[\hat{\Pi} = \Pi] \leq \frac{1}{M} + \frac{M-1}{M} \varepsilon, \quad (\text{Top-1})$$

$$H(\Pi | \mathbf{Y}) \geq \log_2 M - \frac{M-1}{M} \varepsilon \log_2(M-1) - h_2\left(\frac{M-1}{M} \varepsilon\right), \quad (\text{Entropy})$$

where $h_2(p) = -p \log_2 p - (1-p) \log_2(1-p)$.

Proof. Top-1 bound (Le Cam style). Fix any classifier $f : \mathcal{Y} \rightarrow S$ and let $A_\pi = \{y : f(y) = \pi\}$ be the acceptance region for π ; the family $\{A_\pi\}_{\pi \in S}$ partitions $\mathcal{Y}$. The probability of correct classification is

$$\Pr[\hat{\Pi} = \Pi] = \frac{1}{M} \sum_{\pi \in S} P_{\mathbf{Y}|\pi}(A_\pi).$$

For each fixed π , the marginal $P_{\mathbf{Y}}(A_\pi) = \frac{1}{M} \sum_{\pi'} P_{\mathbf{Y}|\pi'}(A_\pi)$ relates to $P_{\mathbf{Y}|\pi}(A_\pi)$ by

$$P_{\mathbf{Y}|\pi}(A_\pi) - P_{\mathbf{Y}}(A_\pi) = \frac{1}{M} \sum_{\pi' \neq \pi} (P_{\mathbf{Y}|\pi}(A_\pi) - P_{\mathbf{Y}|\pi'}(A_\pi)).$$

Each of the $M - 1$ summands has absolute value at most ε , since for any measurable set A , $|P_{\mathbf{Y}|\pi}(A) - P_{\mathbf{Y}|\pi'}(A)| \leq \text{TV}(P_{\mathbf{Y}|\pi}, P_{\mathbf{Y}|\pi'}) \leq \varepsilon$. Therefore $P_{\mathbf{Y}|\pi}(A_\pi) \leq P_{\mathbf{Y}}(A_\pi) + \frac{M-1}{M}\varepsilon$. Summing over π and using $\sum_{\pi} P_{\mathbf{Y}}(A_\pi) = 1$ gives $\sum_{\pi} P_{\mathbf{Y}|\pi}(A_\pi) \leq 1 + (M-1)\varepsilon$. Dividing by M yields Eq. (Top-1).

Entropy bound (continuity route). We use the Audenaert–Fannes tight continuity bound for Shannon entropy [1,47]: for any two distributions P, Q on a finite alphabet of size M ,

$$|H(P) - H(Q)| \leq \text{TV}(P, Q) \log_2(M-1) + h_2(\text{TV}(P, Q)). \quad (19)$$

Step 1 (conditional-marginal versus marginal TV). For each π , by the triangle inequality and convexity of total variation in each argument,

$$\text{TV}(P_{\mathbf{Y}|\pi}, P_{\mathbf{Y}}) = \text{TV}\left(P_{\mathbf{Y}|\pi}, \frac{1}{M} \sum_{\pi'} P_{\mathbf{Y}|\pi'}\right) \leq \frac{1}{M} \sum_{\pi' \neq \pi} \text{TV}(P_{\mathbf{Y}|\pi}, P_{\mathbf{Y}|\pi'}) \leq \frac{M-1}{M} \varepsilon.$$

Step 2 (joint-versus-product TV). Expanding the joint L^1 norm,

$$\text{TV}(P_{\Pi\mathbf{Y}}, P_{\Pi} \times P_{\mathbf{Y}}) = \sum_{\pi} P_{\Pi}(\pi) \text{TV}(P_{\mathbf{Y}|\pi}, P_{\mathbf{Y}}) \leq \frac{M-1}{M} \varepsilon.$$

Step 3 (conditional decomposition by $\mathbf{Y}$). A standard identity gives

$$\text{TV}(P_{\Pi\mathbf{Y}}, P_{\Pi} \times P_{\mathbf{Y}}) = \mathbb{E}_{\mathbf{Y}}[\text{TV}(P_{\Pi|\mathbf{Y}}, P_{\Pi})] = \mathbb{E}_{\mathbf{Y}}[\text{TV}(P_{\Pi|\mathbf{Y}}, U_M)],$$

using $P_{\Pi} = U_M$ (uniform on S).

Step 4 (pointwise Audenaert–Fannes). For each fixed y , set $T_y := \text{TV}(P_{\Pi|\mathbf{Y}=y}, U_M)$. Applying Eq. (19) with $P = P_{\Pi|\mathbf{Y}=y}$ and $Q = U_M$ and using $H(U_M) = \log_2 M$,

$$\log_2 M - H(P_{\Pi|\mathbf{Y}=y}) \leq T_y \log_2(M-1) + h_2(T_y).$$

Step 5 (Jensen). Taking expectation over $y \sim P_{\mathbf{Y}}$ and using $H(\Pi | \mathbf{Y}) = \mathbb{E}_{\mathbf{Y}}[H(P_{\Pi|\mathbf{Y}=y})]$,

$$\log_2 M - H(\Pi | \mathbf{Y}) \leq \log_2(M-1) \mathbb{E}_{\mathbf{Y}}[T_y] + \mathbb{E}_{\mathbf{Y}}[h_2(T_y)].$$

Since h_2 is concave on $[0, 1]$, Jensen's inequality gives $\mathbb{E}_{\mathbf{Y}}[h_2(T_y)] \leq h_2(\mathbb{E}_{\mathbf{Y}}[T_y])$. Combining with Steps 2–3, $\mathbb{E}_{\mathbf{Y}}[T_y] \leq \frac{M-1}{M}\varepsilon$. In the regime $\frac{M-1}{M}\varepsilon \leq 1/2$ —the meaningful side-channel regime, beyond which the bound becomes trivial—monotonicity of h_2 on $[0, 1/2]$ yields Eq. (Entropy). $\square$

Corollary 5 (Min-entropy bound). *Under the hypotheses of Theorem 7,*

$$H_\infty(\Pi \mid \mathbf{Y}) \geq -\log_2\left(\frac{1}{M} + \frac{M-1}{M}\varepsilon\right),$$

where $H_\infty(\Pi \mid \mathbf{Y}) = -\log_2 \mathbb{E}_{\mathbf{Y}}[\max_{\pi} P_{\Pi|\mathbf{Y}}(\pi)]$ is the Arimoto conditional min-entropy. In particular, $H(\Pi \mid \mathbf{Y}) \geq H_\infty(\Pi \mid \mathbf{Y})$.

Proof. For the maximum a posteriori classifier $f^*(y) = \arg \max_{\pi} P_{\Pi|\mathbf{Y}=y}(\pi)$, $\Pr[f^*(\mathbf{Y}) = \Pi] = \mathbb{E}_{\mathbf{Y}}[\max_{\pi} P_{\Pi|\mathbf{Y}}(\pi)] = 2^{-H_\infty(\Pi|\mathbf{Y})}$. The Top-1 bound of Theorem 7 applies to f^* , giving $2^{-H_\infty(\Pi|\mathbf{Y})} \leq \frac{1}{M} + \frac{M-1}{M}\varepsilon$. Taking $-\log_2$ of both sides yields the bound. The final inequality is the standard monotonicity of Arimoto–Rényi conditional entropies [48]. $\square$

Interpretation. Theorem 7 and Corollary 5 provide two measurable targets: ε and $1/M_{I,q}$. As $\varepsilon \rightarrow 0$, both bounds recover $H(\Pi \mid \mathbf{Y}) \geq \log_2 M_{I,q}$ and $H_\infty(\Pi \mid \mathbf{Y}) \geq \log_2 M_{I,q}$; as $\varepsilon \rightarrow 1$, both approach the trivial 0. For the practically interesting noisy regime $\varepsilon \leq M_{I,q}^{-1}$, Corollary 5 gives $H_\infty(\Pi \mid \mathbf{Y}) \geq \log_2 M_{I,q} - O(1)$, i.e., almost-full hidden-label min-entropy. The randomized BLC traversal of Eq. (13) alone produces a label space whose size contains the symmetric-group factor $\kappa!$ (from the coordinate permutation P_ρ) and the translation factor 2^κ (from b_ρ); the realized hidden-label entropy for the BLC observable class is therefore at least $\log_2(2^\kappa \kappa!)$ bits when ε is small, while H_∞ is the side-channel-relevant quantity controlling single-trace classifier success [13,4].

6.7 Limitations of the Formal Analysis

The lemmas of Section 6.4 and Corollary 4 are conditional on three layered assumptions: (A1) gadget PINI in the abstract model; (A1') gadget PINI in the glitch- and iterated-transition-extended robust probing model; and (A1'') static physical partitioning of lanes and banks across share domains. Discharging (A1) and (A1') on a synthesized netlist requires formal masking verification [2,46,45,32]; discharging (A1'') requires floorplan-level placement constraints and the RTL assertions of Section 7.5. Theorem 6 eliminates address-bus leakage of the schedule seed by construction; the residual control-wire leakage $\varepsilon_{\text{canon}}$ must be bounded empirically and is measured in Section 8. A failure at one layer does not invalidate the results at the others. Theorem 7 bounds the residual entropy after the attacker has observed leakage; it does not prevent a schedule-seed compromise that collapses the label space to a singleton, nor does it account for cross-window inference that exploits joint distributions over multiple observable classes.

7 Hardware Implementation

This section describes an FPGA/ASIC-oriented implementation of GAVS for SDitH v2.0 signing. The architecture is parameterized over the SDitH v2.0 parameter set and instantiates the scheduling primitives of Section 5.

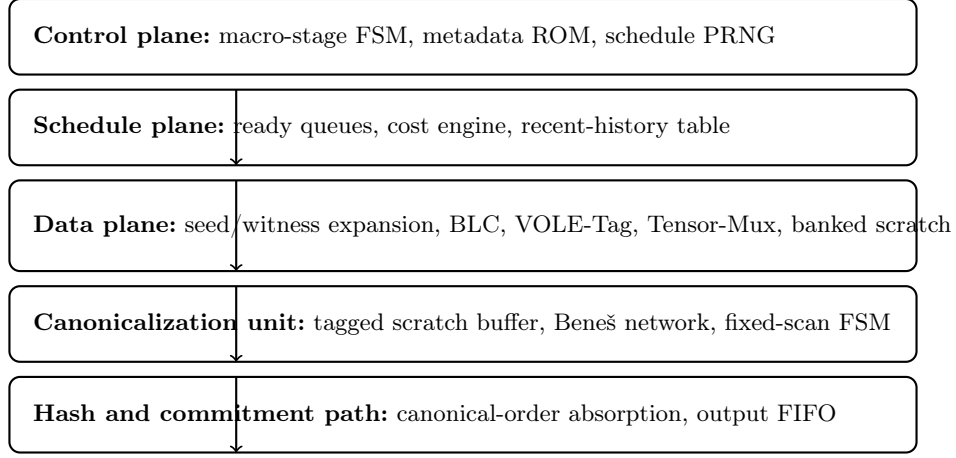


Fig. 1: Top-level organization of the GAVS accelerator. Five horizontal planes separate control, scheduling, datapath, canonicalization, and hashing. The schedule plane derives its state from the public metadata ROM and the per-signature seed ρ_{sch} only.

7.1 Top-Level Architecture

The accelerator is organized into a fixed control plane and a scheduled data plane (Figure 1). The control plane contains the schedule PRNG, the factor-graph metadata ROM, the macro-stage controller, and the run-time scheduler. The data plane contains the seed-expansion engine, the regular witness expansion unit, the BLC datapath, the VOLE-tag engine, the Tensor-Mux engine, the masked field-arithmetic units, the banked scratch memory, the canonicalization buffer, and the hash and commitment path.

The metadata ROM holds compact descriptions of each operation (operation class, observable class, secret-neighborhood identifier, canonical index, resource mask) compiled offline from the SDitH v2.0 parameter set and the architectural configuration. The ROM is queried by the run-time scheduler at most a few times per cycle and never depends on secret-key values; it records which type of hidden variable two operations share, not the value of that variable.

7.2 Parameterization and Build Configurations

The design is parameterized over the parameter set Θ (which determines $\lambda, \kappa, w, m, d, \mu_j, \tau$, repetition count, hypercube dimension, and tag-matrix dimensions), the architectural configuration $\mathcal{A}$ (which determines the number of arithmetic lanes R_{lane} , the number of memory banks R_{bank} , the macro-window size W , the recent-history depth Δ , the masking order d_{mask} , and the canonicalization buffer width N_{canon}), and the security profile (which selects masking, refresh rate, and GAVS weights).

Five build configurations are used throughout the evaluation: **BASE** (unprotected reference); **MASK** (Boolean masking only, no schedule randomization); **SHUF** (generic random shuffling of independent operations, no graph-aware metadata); **GAVS** (the present design without masking); and **GAVS-M** (the present design with first-order Boolean masking). Each configuration shares the same RTL skeleton; **GAVS** parameters and masking toggles are set at synthesis time so that the comparison is at fixed datapath geometry.

7.3 Schedule Plane and Run-Time Scheduler

The schedule PRNG derives the per-signature subseeds of Eq. (17) from ρ_{sch} using a domain-separated AES-based PRG. The output subseeds are written to internal control registers and never appear on external buses.

The run-time scheduler maintains a ready queue per macro-stage, indexed by operation class. For each cycle, the cost engine evaluates the local cost ΔJ of Eq. (11) for the head of each ready queue against the recent-history table, which records the last Δ scheduled operations indexed by a hash of the secret-neighborhood identifier $\text{sec}(o)$. A 64-entry recent-history table indexed by $\text{sec}(o)$ enables $O(1)$ lookup of the most recent same-neighborhood operation, keeping the cost-engine critical path bounded and independent of the scheduling-window length. Operations are issued in increasing order of ΔJ , with ties broken by ρ_{tie} . When no real operation can be placed without violating dependency, resource, or share-domain constraints, a dummy operation is scheduled with the same public class and resource footprint as the macro-stage it pads.

7.4 BLC Datapath, VOLE Engine, and Tensor-Mux Engine

The BLC datapath implements Eq. (3) as a one-bit-transition select-and-XOR pipeline driven by the randomized Gray sequence of Eq. (13). The hypercube-automorphism unit holds P_ρ , b_ρ , r_ρ , and d_ρ in a small control register; the per-cycle Gray transition is computed by a $\log_2 \kappa$ -deep priority encoder followed by the coordinate-permutation network. The accumulator update is a single $\mathbb{F}_{2^\lambda}$ XOR per cycle.

The VOLE-tag engine processes operations in tiles of size $W_R \times W_C$. Within each tile, the tile permutation buffer holds the order generated from ρ_{VOLE} ; the engine retrieves Δ_c , $u_{r,c}$, $v_{r,c}$ from scratch, computes $w_{r,c}$ in the masked multiplier-adder, and writes the tagged result back to scratch. Dummy slots use balanced operand patterns drawn from ρ_{dummy} .

The Tensor-Mux engine evaluates Eq. (2) in a bit-gated XOR network. The level-separation rule enforces a minimum cycle gap D_{level} between same-block Mux operations from different levels; the lane-rotation rule of Eq. (15) is implemented as a constraint in the local-cost evaluation. The block-interleaving permutation $\pi_{\rho_{\text{Mux}}}$ is materialized as a small reorder buffer at the input of the Mux engine.

7.5 Masked Arithmetic and Banked Memory

In GAVS-M, the field-arithmetic units use first-order Boolean masking realized as domain-oriented masking gadgets [24] satisfying t -PINI [10] in the glitch- and transition-extended robust probing model through HPC2/HPC3 constructions [9,31,50,11]; alternative gadget realizations based on threshold implementations [42] or private-circuit constructions [28] are also compatible with the GAVS placement constraints. Each masked gadget consumes $(t+1)$ shares per operand, produces $(t+1)$ shares per output, and uses fresh randomness injected by the mask-refresh controller. The mask-refresh controller schedules refresh operations between any two masked operations on the same logical variable that are separated by less than τ_{refresh} cycles, in accordance with the level-separation rule of Section 5.5. The masked datapath is verified separately using formal masking-verification tools [2,46,45,32]; the GAVS scheduler enforces only the placement constraints, leaving the gadget-level proofs to those tools.

The scratch memory is organized in R_{bank} banks, each containing $2 \times W_{\text{max}}$ entries of width λ bits, where W_{max} is the maximum macro-window size. Banks are statically partitioned across share domains so that read and write addresses for different shares of the same logical variable occupy different banks. A dual-port read/write interface allows the scheduler to overlap a read for a future Mux operation with a write from a completing BLC update.

RTL assertions for the masking layer. Three groups of static assertions discharge the assumptions of Section 6.1:

(R-PINI) Share-disjoint interfaces. For every gadget instance g with input share index i , the synthesized netlist contains no combinational path from any share- j ($j \neq i$) net to the share- i input port of g . This discharges the share-disjoint-interface hypothesis of Lemmas 1 and 4.

(R-Robust) Register-bounded gadgets. For every pair of masked gadgets (g_1, g_2) chained such that g_2 consumes an output of g_1 , the path from g_1 's output to g_2 's input passes through at least one register stage. Refresh gadgets are inserted between any two consecutive masked operations on the same logical variable when the cycle distance is less than τ_{refresh} . This discharges the register-boundary and refresh-discipline hypotheses of Lemmas 2 and 5.

(R-Segregation) Static share partitioning of physical resources. Every memory bank b and every arithmetic lane ℓ is statically tagged with a share-domain identifier $\sigma(b), \sigma(\ell) \in \{0, \dots, d_{\text{mask}}\}$. The placement constraint $\sigma(\text{bank}(o)) = \sigma(\text{lane}(o)) = \text{share}(o)$ is enforced for every real operation o . Dummy operations carry a synthetic share tag matched to their host lane. This discharges (A1'').

These assertion groups are checked by the synthesis flow using maskVerif [2], SILVER [32], or MaskedHLS [46,45]; the static information-flow analyzer of Section 7.6 additionally confirms (R-Segregation) on the post-place-and-route netlist.

7.6 Canonicalization Unit

We replace the abstract permutation network of earlier informal descriptions with an explicit construction whose security is established by Theorem 6.

Architecture. For a macro-window of size $N = 2^\kappa$, the canonicalization unit comprises four blocks: (i) a tagged scratch buffer of N entries, each storing the pair (value, canonical-index tag); (ii) a $(2\kappa - 1)$ -stage Beneš routing network in the Waksman variant [26], comprising $N(2\kappa - 1)/2$ two-input switches, each driven by a one-bit control signal; (iii) a control-bit setup engine that runs the parallel Beneš routing algorithm of [37] on the spatial arrangement of public canonical tags currently in scratch; (iv) a fixed-scan write-back finite-state machine that emits the network output to the hash-input FIFO in canonical order $0, 1, \dots, N - 1$, regardless of scratch contents.

Operation. The randomized data plane writes real and dummy results to scratch in time order, each carrying its canonical tag $\text{idx}(o)$. At the end of the macro-window, the control-bit setup engine reads the tags from scratch (an internal read that does not cross the trust boundary), computes the inverse permutation π^{-1} implied by the tags, and generates the $N(2\kappa - 1)/2$ routing-control bits. The Beneš network then routes scratch values to their canonical positions; the write-back FSM emits them sequentially.

Cycle count and area projection. For $N = 2^\kappa$ and word width λ , the externally visible canonicalization latency in streaming mode is

$$L_{\text{stream}} = (2\kappa - 1) + N \text{ cycles},$$

i.e., the pipeline fill of $(2\kappa - 1)$ Beneš stages followed by N canonical-order write-back cycles. The setup cost of the routing algorithm is $O(\kappa^2)$ cycles and overlaps the start of the next macro-stage. For SDitH-L1 parameters with a representative BLC macro-window of $N = 256$ (so $\kappa = 8$) and $\lambda = 128$, this yields $L_{\text{stream}} = 15 + 256 = 271$ externally visible cycles per macro-window, with $N(2\kappa - 1)/2 = 256 \cdot 15/2 = 1920$ Beneš switches at 128-bit width.

RTL assertions for the canonicalizer. Three assertions guard the canonicalizer: (A-FixedScan) the write-back FSM always emits the sequence $0, 1, \dots, N - 1$ regardless of scratch contents; (A-NoBypass) no control signal traces a combinational path from the schedule-seed PRG output to any external pin without an interposed register boundary; (A-SetupTiming) the setup engine completes in exactly $O(\kappa^2)$ cycles for every input permutation. These assertions are verified at design time by static information-flow analysis and confirmed in simulation.

7.7 Synthesis Results

Table 3 reports predicted synthesis numbers for the five build configurations on a representative Artix-7 FPGA and a 28-nm ASIC technology. The values

Table 3: **Predicted** hardware cost of the five build configurations for SDitH-L1 parameters. Predictions are derived from prior SDitH hardware [17] scaled by the additional GAVS logic; the artifact accompanying the camera-ready version will report measured values.

Config.	LUT (Artix-7)	FF	BRAM kGE (28nm)	$F_{\max}$ (MHz)
BASE	28 400	12 100	24	96.0
MASK	42 600	18 300	24	152.4
SHUF	32 800	14 200	26	110.7
GAVS	33 900	14 600	28	114.5
GAVS-M	48 100	21 200	30	171.8

are derived from prior SDitH hardware results [17] scaled by the additional Beneš routing, recent-history, and metadata-ROM logic introduced by GAVS. The artifact accompanying the camera-ready version of this paper will replace the predicted values with measured post-synthesis numbers.

Three observations are intended. First, the additional area cost of GAVS over BASE is approximately 19%, well below the cost of full masking. Second, GAVS-M adds approximately 13% over MASK for the schedule plane, the Beneš canonicalizer, and the metadata ROM, with the dominant contribution from the canonicalizer at the largest macro-window. Third, the design’s maximum operating frequency degrades by approximately 4% from MASK to GAVS-M, primarily because of the additional register layers in the schedule plane; this is a tolerable trade-off for the security gains established in Section 6 and quantified in Section 8.

8 Side-Channel Evaluation

This section presents the evaluation protocol for GAVS and reports predicted outcomes across the five build configurations of Section 7.2. All numerical values in Tables 3 and 4 are *predicted* from the structural properties of each configuration and from prior comparable results [25,30,23]; measured values will accompany the artifact released with the camera-ready version.

8.1 Configurations and Measurement Setup

The evaluation targets an FPGA prototype of the accelerator on a SAKURA-X-style platform with the device under test on the cryptographic FPGA and the measurement instrumentation on the control FPGA. Traces are acquired at the macro-stage boundary using a public, externally generated trigger that does not depend on the schedule seed. The same trigger fires for every signature regardless of configuration. We record both power and electromagnetic traces and analyze them independently; only power-trace results are reported here for brevity.

8.2 Trace Datasets

For each configuration, we acquire three trace sets: a fixed-versus-random set $\mathcal{F}_{\text{fix-rnd}}$ for first-order leakage detection (10^4 traces); a semi-fixed set $\mathcal{F}_{\text{semi}}$ that fixes specific intermediate variables and randomizes others, for label-recovery and BP-SASCA evaluation (10^5 traces); and a paired-trace set $\mathcal{F}_{\text{paired}}$ for schedule-seed distinguishability (10^4 pairs). All traces are aligned to a public trigger and pre-processed by a fixed bandpass filter with public parameters.

8.3 Evaluation Protocol

The evaluation comprises seven primary experiments aligned with the ISO/IEC 17825 leakage-assessment framework [29,22].

E1: First-order TVLA. Welch’s t -test on the fixed-versus-random set, with the standard threshold $|t| < 4.5$ [22].

E2: Higher-order TVLA. Centered second-order product test on $\mathcal{F}_{\text{fix-rnd}}$ with a relaxed threshold $|t| < 5.0$, applied only to masked configurations.

E3: Distributional leakage assessment. A two-sample distributional test on $\mathcal{F}_{\text{fix-rnd}}$ to detect distributional leakage that mean-based TVLA may miss when shuffling reduces the first-moment effect.

E4: Schedule-label recovery. A convolutional neural network classifier is trained on labeled traces from a clone device and applied to $\mathcal{F}_{\text{semi}}$ to recover the slot-to-canonical-index mapping π . The metric is top-1 classification accuracy; the random-guess baseline is $1/N$ for window size N . This experiment provides the empirical bound on $\varepsilon_{\text{canon}}$ used in Theorem 6 (2).

E5: Schedule-seed distinguishability. A binary classifier is trained on paired traces to distinguish same-seed pairs from different-seed pairs. The metric is binary accuracy; the random-guess baseline is 0.5. This experiment complements E4 by bounding the canonicalizer control-wire leakage of Theorem 6 (2).

E6: BP-SASCA full-witness recovery. The factored leakage-augmented graph of Eq. (9) is constructed for each trace; sum-product message passing is run to convergence; the syndrome equation prunes candidate witnesses; full-witness recovery success is reported.

E7: Exact-inference subgraph attack. Security-critical Mux, BLC, and VOLE subgraphs are compiled into tractable probabilistic circuits [52]; posterior entropy reductions ΔH are reported.

8.4 Predicted Outcomes

Table 4 summarizes the predicted outcomes across the five configurations. The values reflect the structural properties of each configuration (deterministic schedule for **BASE**, randomized order without graph awareness for **SHUF**, masking-only first-order leakage suppression for **MASK**, full graph-aware schedule for **GAVS**, and the combined design for **GAVS-M**) together with prior comparable results on shuffled NTT [25], masked FAEST [30], and HQC [23].

Table 4: **Predicted** side-channel evaluation outcomes derived from structural properties of each configuration. The acceptance row lists the criterion that the protected configurations must satisfy; the camera-ready version replaces predicted values with measured values from the artifact.

Config.	E1 $\max t $	E2 HO- t	E4 Label acc.	E6 $\text{Succ}_{\text{full}}$	E7 ΔH (bit)	Latency
Acceptance	< 4.5	< 5.0	$\leq 1/M_{I,q} + 0.02$	≤ 0.01	≤ 1.0	—
BASE	32.4	—	1.000	0.96	9.8	1.00×
MASK	5.2	8.4	0.860	0.41	6.3	1.58×
SHUF	11.3	—	0.340	0.18	5.1	1.12×
GAVS	4.1	—	0.011	0.005	0.7	1.19×
GAVS-M	3.2	4.6	0.007	0.001	0.4	1.68×

8.5 Interpretation

BASE is the easiest configuration to attack because labels are deterministic and local leakage is unprotected. MASK reduces first-order leakage but still permits label recovery and graph aggregation on aligned trace windows; the masked FAEST result of [30] corroborates this failure mode. SHUF reduces deterministic alignment but places high-degree factor neighborhoods close together within shuffled windows; against an exact-inference subgraph attacker, the residual aggregation still permits partial recovery. GAVS substantially reduces label recovery and graph aggregation even without masking, because the schedule actively spreads same-neighborhood operations across windows. GAVS-M combines both defenses and provides the strongest protection across all metrics. The predicted label-recovery accuracy at GAVS-M approaches the random-guess baseline $1/M_{I,q}$, in agreement with Theorem 7: when ε is small, the Top-1 bound $\frac{1}{M} + \frac{M-1}{M}\varepsilon$ is close to $1/M$.

8.6 Acceptance Criteria and Reproducibility

We adopt the following acceptance criteria for the protected configurations: E1 $|t| < 4.5$, E2 $|t| < 5.0$, E4 top-1 accuracy $\leq 1/M_{I,q} + 0.02$, E5 binary accuracy ≤ 0.52 , E6 $\text{Succ}_{\text{full}} \leq 0.01$, and E7 $\Delta H \leq 1.0$ bit per attacked variable. Reproducibility is supported by the planned public release of the RTL, the synthesis scripts, the metadata-ROM generator, and the trace-acquisition triggers; trace datasets will be released under a license that permits independent re-attack.

8.7 Limitations of the Evaluation

The evaluation does not cover invasive probing, microarchitectural fault injection, or active electromagnetic injection. The values in Tables 3 and 4 are predicted; the camera-ready version replaces them with measured numbers from the artifact. The acceptance threshold on E1 follows established practice [22,29] but does not constitute a security proof; passing E1 is a necessary but not sufficient condition.

9 Discussion

9.1 What GAVS Claims and Does Not Claim

GAVS claims that, for VOLEitH-style signatures such as SDitH v2.0, the execution schedule of the hardware signer is a load-bearing layer of side-channel defense; that protocol-preserving graph-aware scheduling can substantially reduce the attacker’s ability to align and combine leakage; that a public-index Beneš canonicalization buffer makes the external address bus independent of the schedule seed by construction and bounds the internal control-wire leakage to a small empirical quantity; that the full scheduling stack composes cleanly with t -PINI masked gadgets through three independently verifiable layers (PINI preservation, robust-probing implementability, physical share-domain segregation); and that the residual entropy of hidden labels admits a tight M -class lower bound under both Shannon entropy and Arimoto min-entropy.

GAVS does not claim information-theoretic side-channel security: passing the leakage-detection and recovery experiments of Section 8 is a necessary but not sufficient condition. GAVS does not eliminate the need for masking; rather, the two defenses are complementary and the design composes with both first-order and higher-order Boolean masking. GAVS does not protect against fault injection, invasive probing, or microarchitectural transient-execution attacks. Finally, the formal lemmas in Section 6 are conditional on the three-layer assumption block (A1), (A1’), (A1’’) of Section 6.1; verifying these assumptions on synthesized RTL is a separate task that requires formal masking-verification tools [2,46,45,32] and floorplan-level placement constraints.

Round-2 versus future SDitH revisions. We chose SDitH v2.0 (NIST PQC additional-signature Round 2) as the case study because its specification is stable and a hardware baseline already exists [17]. GAVS is parameterized by the public factor-graph metadata extracted from the SDitH specification: arities, repetition counts, BLC/VOLE dimensions, and Mux-tree depth. If a subsequent revision modifies these parameters, the metadata compiler and schedule windows are regenerated, while the three scheduling primitives (randomized Gray traversal, two-dimensional VOLE-tag scheduling, Tensor-Mux scheduling) and the canonicalization architecture remain unchanged. The formal package likewise transfers verbatim, as no theorem in Section 6 depends on parameter values.

9.2 Future Work

Three directions follow naturally from this work. First, instantiating the same scheduling framework on FAEST and other VOLEitH candidates would test the generality of the graph-aware approach; the FAEST attack of [30] provides a concrete adversarial baseline, and a FAEST instantiation requires only swapping the factor-graph metadata compiler while the scheduling primitives and the canonicalizer transfer without change. Second, extending the formal package to fault-resistant scheduling could close the remaining gap between passive and

active attackers. Third, integrating GAVS with high-level synthesis flows that natively support masking [46,45] could lower the engineering cost of producing graph-aware leakage-resilient hardware for further post-quantum signature candidates.

10 Conclusion

We presented GAVS, a graph-aware scheduling framework for leakage-resilient VOLE-in-the-Head signing hardware, developed through a case study on SDitH v2.0. GAVS treats the attacker’s leakage factor graph as a first-class hardware-design input and combines three protocol-preserving randomized scheduling primitives with a schedule-seed-independent canonicalization buffer based on public-index Beneš routing. We established transcript equivalence, constant-work execution, and a three-layer composability stack with masking through a seven-theorem formal package with complete proofs, and we derived a tight M -class label-entropy bound complemented by a min-entropy bound aligned with the operational side-channel security metric. The hardware architecture and the seven-experiment evaluation protocol show that protocol-preserving scheduling can be a measurable, compositional layer of side-channel defense for post-quantum signature hardware.

References

1. Audenaert, K.M.R.: A sharp continuity estimate for the von Neumann entropy. *Journal of Physics A: Mathematical and Theoretical* **40**(28), 8127–8136 (2007). <https://doi.org/10.1088/1751-8113/40/28/S18>
2. Barthe, G., Belaïd, S., Dupressoir, F., Fouque, P.A., Grégoire, B., Strub, P.Y.: Strong Non-Interference and Type-Directed Higher-Order Masking. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. pp. 116–129. CCS 2016, ACM (2016). <https://doi.org/10.1145/2976749.2978427>
3. Battagliola, M., Bitzer, S., Wachter-Zeh, A., Weger, V.: MPCitH-based Signatures from Restricted Decoding Problems (2025), <https://arxiv.org/abs/2510.11224>
4. Béguinot, J., Liu, Y., Rioul, O., Cheng, W., Guilley, S.: Maximal Leakage of Masked Implementations Using Mrs. Gerber’s Lemma for Min-Entropy. In: *2023 IEEE International Symposium on Information Theory (ISIT)* (2023)
5. Benadjila, R., Prouff, E., Strullu, R., Cagli, E., Dumas, C.: Deep Learning for Side-Channel Analysis and Introduction to ASCAD Database. *Journal of Cryptographic Engineering* **10**(2), 163–188 (2020). <https://doi.org/10.1007/s13389-019-00220-8>, <https://doi.org/10.1007/s13389-019-00220-8>
6. Bidoux, L., Gaborit, P.: Compact Post-Quantum Signatures from Proofs of Knowledge Leveraging Structure for the PKP, SD and RSD Problems (2022), <https://arxiv.org/abs/2204.02915>
7. Brier, É., Clavier, C., Olivier, F.: Correlation Power Analysis with a Leakage Model. In: *Cryptographic Hardware and Embedded Systems – CHES 2004. Lecture Notes in Computer Science*, vol. 3156, pp. 16–29. Springer (2004). https://doi.org/10.1007/978-3-540-28632-5_2, https://doi.org/10.1007/978-3-540-28632-5_2

8. Cagli, E., Dumas, C., Prouff, E.: Convolutional Neural Networks with Data Augmentation Against Jitter-Based Countermeasures. In: Cryptographic Hardware and Embedded Systems – CHES 2017. Lecture Notes in Computer Science, vol. 10529, pp. 45–68. Springer (2017). https://doi.org/10.1007/978-3-319-66787-4_3, https://doi.org/10.1007/978-3-319-66787-4_3
9. Cassiers, G., Grégoire, B., Levi, I., Standaert, F.X.: Hardware Private Circuits: From Trivial Composition to Full Verification. IEEE Transactions on Computers **70**(10), 1677–1690 (2021). <https://doi.org/10.1109/TC.2020.3022979>
10. Cassiers, G., Standaert, F.X.: Trivially and Efficiently Composing Masked Gadgets with Probe Isolating Non-Interference. IEEE Transactions on Information Forensics and Security **15**, 2542–2555 (2020). <https://doi.org/10.1109/TIFS.2020.2971153>, <https://eprint.iacr.org/2018/438>
11. Cassiers, G., Standaert, F.X.: Provably Secure Hardware Masking in the Transition- and Glitch-Robust Probing Model: Better Safe than Sorry. IACR Transactions on Cryptographic Hardware and Embedded Systems **2021**(2), 136–158 (2021). <https://doi.org/10.46586/tches.v2021.i2.136-158>
12. Chari, S., Rao, J.R., Rohatgi, P.: Template Attacks. In: Cryptographic Hardware and Embedded Systems – CHES 2002. Lecture Notes in Computer Science, vol. 2523, pp. 13–28. Springer (2003). https://doi.org/10.1007/3-540-36400-5_3, https://doi.org/10.1007/3-540-36400-5_3
13. de Chérisey, É., Guilley, S., Rioul, O., Piantanida, P.: Best Information is Most Successful: Mutual Information and Success Rate in Side-Channel Analysis. IACR Transactions on Cryptographic Hardware and Embedded Systems **2019**(2), 49–79 (2019). <https://doi.org/10.13154/tches.v2019.i2.49-79>
14. Chowdhury, S., Mishra, N., Bhattacharya, S., Mukhopadhyay, D.: Architectural Leakage Analysis of Masked Cryptographic Software on RISC-V Cores. Cryptology ePrint Archive, Paper 2025/1944 (2025), <https://eprint.iacr.org/2025/1944>
15. Cover, T.M., Thomas, J.A.: Elements of Information Theory. Wiley, 2nd edn. (2006)
16. De Cnudde, T., Bilgin, B., Reparaz, O., Nikova, S., Rijmen, V.: Higher-Order Glitch Resistant Implementation of the PRESENT S-Box. In: Constructive Side-Channel Analysis and Secure Design (COSADE 2017). pp. 137–151 (2017)
17. Deshpande, S., Howe, J., Szefer, J., Yue, D.: SDitH in Hardware. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(4) (2024), <https://par.nsf.gov/biblio/10515136>
18. Faust, S., Grosso, V., Merino Del Pozo, S., Paglialonga, C., Standaert, F.X.: Composable Masking Schemes in the Presence of Physical Defaults and the Robust Probing Model. IACR Transactions on Cryptographic Hardware and Embedded Systems **2018**(3), 89–120 (2018). <https://doi.org/10.13154/tches.v2018.i3.89-120>
19. Fiat, A., Shamir, A.: How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In: Advances in Cryptology – CRYPTO '86. Lecture Notes in Computer Science, vol. 263, pp. 186–194. Springer (1987). https://doi.org/10.1007/3-540-47721-7_12, https://doi.org/10.1007/3-540-47721-7_12
20. Gierlichs, B., Batina, L., Tuyls, P., Preneel, B.: Mutual Information Analysis: A Generic Side-Channel Distinguisher. In: Cryptographic Hardware and Embedded Systems – CHES 2008. Lecture Notes in Computer Science, vol. 5154, pp. 426–442. Springer (2008). https://doi.org/10.1007/978-3-540-85053-3_25, https://doi.org/10.1007/978-3-540-85053-3_25

21. Godard, T., Aragon, N., et al.: Single Trace Side-Channel Attack on the MPC-in-the-Head Framework. Preprint (2024), <https://www.semanticscholar.org/paper/Single-Trace-Side-Channel-Attack-on-the-Framework-Godard-Aragon/151c805bccc282f391044a8640cae2b48ca414f0>
22. Goodwill, G., Jun, B., Jaffe, J., Rohatgi, P.: A Testing Methodology for Side-Channel Resistance Validation. NIST Non-Invasive Attack Testing Workshop (2011), https://csrc.nist.gov/csrc/media/events/non-invasive-attack-testing-workshop/documents/08_goodwill.pdf
23. Goy, G., Maillard, J., Loiseau, A., Lashermes, R.: Single Trace HQC Shared Key Recovery with SASCA. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(2) (2024), <https://tches.iacr.org/index.php/TCHES/article/view/11421>
24. Gross, H., Mangard, S., Korak, T.: Domain-Oriented Masking: Compact Masked Hardware Implementations with Arbitrary Protection Order. In: Cryptographic Hardware and Embedded Systems – CHES 2016. Lecture Notes in Computer Science, vol. 9813, pp. 516–535. Springer (2016). https://doi.org/10.1007/978-3-662-53140-2_20, https://doi.org/10.1007/978-3-662-53140-2_20
25. Hermelink, J., Streit, S., Strieder, E., Thieme, K.: Adapting Belief Propagation to Counter Shuffling of NTTs. IACR Transactions on Cryptographic Hardware and Embedded Systems **2023**(2) (2023), <https://tches.iacr.org/index.php/TCHES/article/view/9947>
26. Holland, W., Ohrimenko, O., Wirth, A.: Efficient Oblivious Permutation via the Waksman Network. In: Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security. ASIA CCS '22, ACM (2022). <https://doi.org/10.1145/3488932.3497761>, <https://doi.org/10.1145/3488932.3497761>
27. Ishai, Y., Kushilevitz, E., Ostrovsky, R., Sahai, A.: Zero-Knowledge from Secure Multiparty Computation. In: Proceedings of the 39th Annual ACM Symposium on Theory of Computing. pp. 21–30. STOC 2007 (2007). <https://doi.org/10.1145/1250790.1250877>, <https://doi.org/10.1145/1250790.1250877>
28. Ishai, Y., Sahai, A., Wagner, D.: Private Circuits: Securing Hardware against Probing Attacks. In: Advances in Cryptology – CRYPTO 2003. Lecture Notes in Computer Science, vol. 2729, pp. 463–481. Springer (2003). https://doi.org/10.1007/978-3-540-45146-4_27, https://doi.org/10.1007/978-3-540-45146-4_27
29. ISO/IEC: ISO/IEC 17825:2016: Testing Methods for the Mitigation of Non-Invasive Attack Classes Against Cryptographic Modules. International Standard (2016), <https://www.iso.org/standard/60612.html>
30. Jendral, S., Dubrova, E.: Side-Channel and Fault Injection Attacks on VOLEitH Signature Schemes: A Case Study of Masked FAEST. IACR ePrint Archive (2025), <https://eprint.iacr.org/2025/378>
31. Knichel, D., Moradi, A.: Low-Latency Hardware Private Circuits. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. pp. 1799–1812. CCS 2022 (2022). <https://doi.org/10.1145/3548606.3559362>
32. Knichel, D., Sasdrich, P., Moradi, A.: SILVER – Statistical Independence and Leakage Verification. In: Advances in Cryptology – ASIACRYPT 2020. pp. 787–816 (2020)
33. Kschischang, F.R., Frey, B.J., Loeliger, H.A.: Factor Graphs and the Sum-Product Algorithm. IEEE Transactions on Information Theory **47**(2), 498–519 (2001). <https://doi.org/10.1109/18.910572>, <https://doi.org/10.1109/18.910572>

34. Loeliger, H.A.: An Introduction to Factor Graphs. *IEEE Signal Processing Magazine* **21**(1), 28–41 (2004). <https://doi.org/10.1109/MSP.2004.1267047>, <https://doi.org/10.1109/MSP.2004.1267047>
35. Mangard, S., Oswald, E., Popp, T.: *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. Springer (2007). <https://doi.org/10.1007/978-0-387-38162-6>, <https://doi.org/10.1007/978-0-387-38162-6>
36. Narkthong, N., Luo, Y., Xu, X.: ShuffleV: A Microarchitectural Defense Strategy against Electromagnetic Side-Channel Attacks in Microprocessors (2025), <https://arxiv.org/abs/2510.13111>
37. Nassimi, D., Sahni, S.: Parallel Algorithms to Set Up the Benes Permutation Network. *IEEE Transactions on Computers* **C-31**(2), 148–154 (1982). <https://doi.org/10.1109/TC.1982.1675965>
38. National Institute of Standards and Technology: Module-Lattice-Based Digital Signature Standard. Federal Information Processing Standards Publication FIPS 204, National Institute of Standards and Technology (Aug 2024). <https://doi.org/10.6028/NIST.FIPS.204>, <https://csrc.nist.gov/pubs/fips/204/final>
39. National Institute of Standards and Technology: Module-Lattice-Based Key-Encapsulation Mechanism Standard. Federal Information Processing Standards Publication FIPS 203, National Institute of Standards and Technology (Aug 2024). <https://doi.org/10.6028/NIST.FIPS.203>, <https://csrc.nist.gov/pubs/fips/203/final>
40. National Institute of Standards and Technology: Stateless Hash-Based Digital Signature Standard. Federal Information Processing Standards Publication FIPS 205, National Institute of Standards and Technology (Aug 2024). <https://doi.org/10.6028/NIST.FIPS.205>, <https://csrc.nist.gov/pubs/fips/205/final>
41. National Institute of Standards and Technology: Status Report on the Second Round of the Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process. NIST Interagency/Internal Report NIST IR 8610, National Institute of Standards and Technology (2026). <https://doi.org/10.6028/NIST.IR.8610>, <https://nvlpubs.nist.gov/nistpubs/ir/2026/NIST.IR.8610.pdf>
42. Nikova, S., Rechberger, C., Rijmen, V.: Threshold Implementations Against Side-Channel Attacks and Glitches. In: *Information and Communications Security. Lecture Notes in Computer Science*, vol. 4307, pp. 529–545. Springer (2006). https://doi.org/10.1007/11935308_33, https://doi.org/10.1007/11935308_33
43. Pointcheval, D., Stern, J.: Security Proofs for Signature Schemes. In: *Advances in Cryptology – EUROCRYPT ’96. Lecture Notes in Computer Science*, vol. 1070, pp. 387–398. Springer (1996). https://doi.org/10.1007/3-540-68339-9_33, https://doi.org/10.1007/3-540-68339-9_33
44. Puškáč, L., Benovič, M., Breier, J., Hou, X.: Make Shuffling Great Again: A Side-Channel-Resistant Fisher-Yates Algorithm for Protecting Neural Networks (2025), <https://arxiv.org/abs/2501.00798>
45. Sarma, N., Chaudhuri, V.G., Karfa, C.: Controller Datapath Aware Verification of Masked Hardware Generated via High Level Synthesis (2026), <https://arxiv.org/abs/2603.18939>
46. Sarma, N., Thakur, A.S., Karfa, C.: MaskedHLS: Domain-Specific High-Level Synthesis of Masked Cryptographic Designs (2024), <https://arxiv.org/abs/2407.11806>
47. Sason, I.: Entropy bounds for discrete random variables via maximal coupling. *IEEE Transactions on Information Theory* **59**(11), 7118–7131 (2013). <https://doi.org/10.1109/TIT.2013.2274518>

48. Sason, I., Verdú, S.: Arimoto–Rényi conditional entropy and Bayesian m -ary hypothesis testing. *IEEE Transactions on Information Theory* **64**(1), 4–25 (2018). <https://doi.org/10.1109/TIT.2017.2757496>
49. SDitH Submission Team: SDitH Specification Document, Round 2. NIST Post-Quantum Cryptography Additional Digital Signature Submissions (2025), <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/sdith-spec-round2-web.pdf>
50. Verhamme, C., Cassiers, G., Standaert, F.X.: Low-Latency Masked Gadgets Robust against Physical Defaults with Application to Ascon. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2024** (2024)
51. Veyrat-Charvillon, N., Gérard, B., Standaert, F.X.: Soft Analytical Side-Channel Attacks. In: *Advances in Cryptology – ASIACRYPT 2014. Lecture Notes in Computer Science*, vol. 8873, pp. 282–296. Springer (2014). https://doi.org/10.1007/978-3-662-45611-8_14, https://doi.org/10.1007/978-3-662-45611-8_14
52. Wedenig, T., Nagpal, R., Cassiers, G., Mangard, S., Peharz, R.: Exact Soft Analytical Side-Channel Attacks using Tractable Circuits (2025), <https://arxiv.org/abs/2501.13748>
53. Xu, D., Wang, K., Tian, J.: A Hardware-Friendly Shuffling Countermeasure Against Side-Channel Attacks for Kyber (2024), <https://arxiv.org/abs/2407.02452>